

# NIT3003 IT Capstone Project 1

## Block 4, Semester 1 2026

Submission Date:  
14 June 2026



**VICTORIA  
UNIVERSITY**  
Sydney Australia

PROJECT PROPOSAL-COMPLETE

### PROJECT 9:

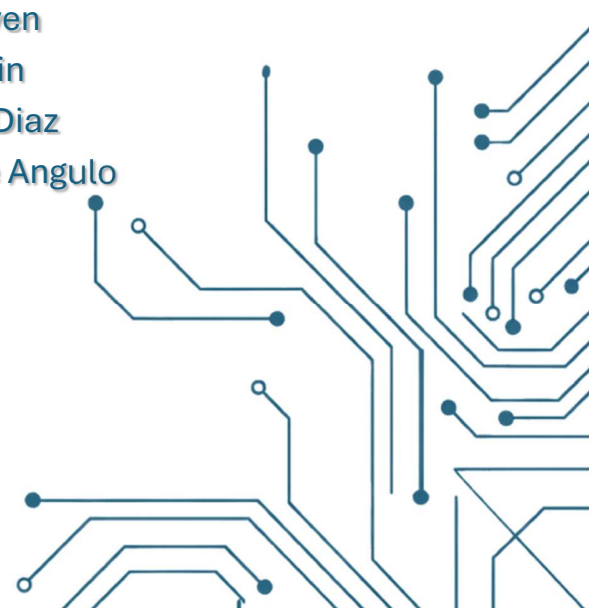
## AI LANGUAGE LEARNING COMPANION (IT, DATA SCIENCE)

#### Team Members:

s8131699 - Quoc Anh, Nguyen  
s8200193 - Htay Hlaing, Win  
s8159573 – Camilo, Salazar Diaz  
s8187769 - Jeison Andres, Aguirre Angulo

#### Block Convenor:

Dr Sardar Farhad



|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>1. Introduction - Written by Jeison</b>                                 | <b>1</b>  |
| <b>1.1. Background and Problem Overview.</b>                               | <b>1</b>  |
| <b>1.2. Proposed Solution</b>                                              | <b>2</b>  |
| <b>1.3. Unique Value Proposition</b>                                       | <b>2</b>  |
| <b>2. Competitor Analysis: An Overview - Written by Jeison &amp; Win</b>   | <b>3</b>  |
| <b>2.1. Rosetta Stone - Written by Win</b>                                 | <b>3</b>  |
| <b>2.2. Pimsleur - Written by Win</b>                                      | <b>4</b>  |
| <b>2.3. Duolingo - Written by Jeison</b>                                   | <b>4</b>  |
| <b>2.4. Babbel - Written by Jeison</b>                                     | <b>5</b>  |
| <b>2.5. Summary Table: Competitor Comparison - Written by Jeison</b>       | <b>6</b>  |
| <b>2.6. Key Insights - Written by Jeison</b>                               | <b>6</b>  |
| <b>3. The Gap the Project Fills - Written by Jeison</b>                    | <b>6</b>  |
| <b>4. Project Aim and Objectives - Written by Jeison</b>                   | <b>7</b>  |
| <b>5. Project Scope - Written by Jeison</b>                                | <b>8</b>  |
| <b>5.1. In Scope - Written by Jeison</b>                                   | <b>8</b>  |
| <b>5.2. Out of Scope - Written by Win</b>                                  | <b>9</b>  |
| <b>6. Functional Requirements (FRs) - Written by Win &amp; Quoc Anh</b>    | <b>9</b>  |
| <b>6.1. Frontend Functionality (UI) - Written by Win</b>                   | <b>9</b>  |
| <b>6.2. Server-Side Logic - Written by Win</b>                             | <b>10</b> |
| <b>6.3. Database Layer (PostgreSQL via Supabase) - Written by Quoc Anh</b> | <b>11</b> |
| <b>6.3.1. Tables and fields</b>                                            | <b>11</b> |
| <b>6.3.2. Entity-Relationship</b>                                          | <b>13</b> |
| <b>6.3.3. Indexes for Performance Optimization</b>                         | <b>13</b> |
| <b>6.3.4. Row Level Security (RLS) Policies</b>                            | <b>14</b> |
| <b>6.4. Backend Workflow Overview - Written by Quoc Anh</b>                | <b>14</b> |
| <b>7. Non-Functional Requirements (NFRs) - Written by Camilo</b>           | <b>15</b> |
| <b>8. Cost and Scalability - Written by Jeison</b>                         | <b>18</b> |
| <b>8.1. API Consumption Costs (AUD)</b>                                    | <b>18</b> |

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 8.2. Operational Subscription Scaling Tiers .....                     | 19 |
| 8.3. Architectural Performance & Processing Bottlenecks.....          | 19 |
| 9. Cost-Benefit and Competitive Analysis - Written by Jeison .....    | 20 |
| 9.1. Financial and Pedagogical Comparison .....                       | 20 |
| 9.2. Market Positioning Strategy (Bypassing the Fluency Plateau)..... | 20 |
| 9.3. Return on Investment (ROI) Analysis .....                        | 21 |
| 9.3.1. Financial Modeling Parameters (AUD) .....                      | 21 |
| 9.3.2. Projections and Break-Even Point (12-Month) .....              | 21 |
| 9.3.3. Profitability and Financial Viability .....                    | 22 |
| 10. Proposed Pricing Plans - Written by Jeison.....                   | 22 |
| 10.1. Subscription Tier Structure .....                               | 22 |
| 10.2. Competitive Pricing Alignment.....                              | 23 |
| 10.3. Market Monetization Logic .....                                 | 23 |
| 11. Use Case - Written by Camilo & Quoc.....                          | 23 |
| 11.1. Register / Login (UC1).....                                     | 24 |
| 11.2. Select Scenario (UC2) .....                                     | 25 |
| 11.3. Set Difficulty Level (UC3).....                                 | 25 |
| 11.4. Record Voice Input (UC4) .....                                  | 26 |
| 11.5. Transcribe Speech (STT)(UC5) .....                              | 27 |
| 11.6. Generate AI Response (UC6) .....                                | 27 |
| 11.7. Provide Grammar Feedback (UC7) .....                            | 28 |
| 11.8. Phonetic Pronunciation Assessment (UC8).....                    | 28 |
| 11.9. Play AI Voice Response (UC9).....                               | 29 |
| 11.10. View Progress Dashboard (UC10) .....                           | 30 |
| 11.11. Manage Subscription (UC11) .....                               | 30 |
| 11.12. Enforce Usage Limits (UC12) .....                              | 31 |
| 11.13. Save Session Data (UC13) .....                                 | 31 |
| 11.14. Maintain Conversation Memory (UC14) .....                      | 32 |
| 11.15. Diagram 1 - Authentication & Configuration .....               | 33 |

|                                                                |    |
|----------------------------------------------------------------|----|
| 11.16. Diagram 2 - Conversation Core.....                      | 34 |
| 11.17. Diagram 3 - Persistence & Progress.....                 | 35 |
| 12. Sequence Diagram - Written by Camilo & Quoc.....           | 36 |
| 12.1. Sequence Diagram 1 - Authentication & Configuration..... | 36 |
| 12.2. Sequence Diagram 2 - Conversation Core .....             | 37 |
| 12.3. Sequence Diagram 3 - Persistence & Progress.....         | 38 |
| 13. Pseudocode - Written by Quoc .....                         | 39 |
| 13.1. Authentication and Session Management .....              | 39 |
| 13.1.1. Sign Up .....                                          | 39 |
| 13.1.2. Sign In .....                                          | 39 |
| 13.1.3. Require Auth (API Helper) .....                        | 40 |
| 13.2. Learning Context & Profile Management .....              | 40 |
| 13.2.1. Update Proficiency Profile.....                        | 40 |
| 13.2.2. Initialize Learning Scenario .....                     | 40 |
| 13.3. Core Voice Processing & AI Generation .....              | 40 |
| 13.3.1. Process Audio Input (STT) .....                        | 40 |
| 13.3.2. Generate Conversation & Feedback Analysis .....        | 41 |
| 13.3.3. Voice Generation (TTS) .....                           | 41 |
| 13.4. Storage Policy and History .....                         | 41 |
| 13.4.1. Interaction Logging.....                               | 41 |
| 13.4.2. Fetch Session History.....                             | 42 |
| 13.5. Export & Utilities .....                                 | 42 |
| 13.5.1. Export Learning Report .....                           | 42 |
| 13.5.2. Generate HTMX UI Fragment .....                        | 42 |
| 13.6. API Error Handling & Rate Limiting.....                  | 42 |
| 13.6.1. Enforce Rate Limits .....                              | 42 |
| 13.6.2. Safe External API Calling with Fallback.....           | 43 |
| 13.7. Storage Management & Garbage Collection .....            | 43 |
| 13.7.1. Scheduled Audio Cleanup (CRON Job).....                | 43 |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>14. UI Design - Written by Win &amp; Quoc</b>              | <b>44</b> |
| <b>14.1. Home Page</b>                                        | <b>44</b> |
| <b>14.2. Sign in Page</b>                                     | <b>44</b> |
| <b>14.3. Dashboard Page</b>                                   | <b>45</b> |
| <b>14.4. Scenarios Library Page</b>                           | <b>45</b> |
| <b>14.5. Individual Scenario Section Page</b>                 | <b>46</b> |
| <b>14.6. Profile Page</b>                                     | <b>46</b> |
| <b>15. Risk &amp; Resource Management - Written by Jeison</b> | <b>47</b> |
| <b>15.1. Resource Allocation Framework</b>                    | <b>47</b> |
| <b>15.2. Risk Analysis and Mitigation Matrix</b>              | <b>48</b> |
| <b>16. Timeline &amp; Milestones - Written by Jeison</b>      | <b>49</b> |
| <b>16.1. NIT3003 IT Capstone Project 1</b>                    | <b>49</b> |
| <b>16.2. Project Implementation</b>                           | <b>50</b> |
| <b>16.2.1. Part A - Project Implementation Gantt Chart</b>    | <b>50</b> |
| <b>16.2.2. Part B - Project Implementation Gantt Chart</b>    | <b>51</b> |
| <b>16.3. NIT3004 IT Capstone Project 2</b>                    | <b>52</b> |
| <b>17. Conclusion - Written by Jeison</b>                     | <b>53</b> |
| <b>References:</b>                                            | <b>54</b> |
| <b>Appendix A – Tables</b>                                    | <b>i</b>  |
| <b>Appendix B – Figures</b>                                   | <b>ii</b> |

# 1. Introduction - Written by Jeison

## 1.1. Background and Problem Overview.

### The Illusion of App-Based Fluency

Recent reports in EdTech indicate that despite the global language learning market being valued over **\$50 billion**, approximately **90% of learners abandon their platforms** within the first month. EdTech Innovation Hub (2026). The fundamental reason: **most language apps are designed around a misunderstanding of how languages actually work**. They treat language as a subject to memorize, not a tool to communicate with. HelloTalk Blog (2026).

It is no surprise why language apps have become the ultimate starting line for millions of aspiring bilinguals. Tools like Duolingo, Babbel, Rosetta Stone, and Pimsleur are fantastic for building daily habits, teaching core vocabulary, and giving beginners a clear, structured roadmap to follow. Languages Around the Globe (2026).

But that is where the problem hits. Being fluent is not about keeping a streak alive or checking off lessons; it is about thinking on your feet, understanding people in the moment, and navigating real-world conversations where there is no script.

### The Core Deficiencies of Traditional Apps

Most language learning apps are designed around repetition and completion of metrics. While this keeps users engaged but does not replicate the demands of actual conversation. Mobile Apps are genuinely good at what they are built to do. The trouble is that what they are built to do only covers part of what fluency actually requires. Crucially, mobile apps rarely provide any real speaking or practical listening experience. EdTech Innovation Hub (2026).

Furthermore, traditional apps fail because they lack one critical aspect: **Adaptive Intelligence**: While apps use rigid, mass-market algorithm paths, AI can adapt content in real-time to a student's unique lifestyle, interests, and immediate needs (e.g., focusing on real-time conversation). [5].

### The AI Shift in Language Acquisition

The popularization of ChatGPT and generative artificial intelligence (AI) technologies has prompted a renewed interest in how apps can be designed to enhance students' language-learning experiences. Zhang, C., Meng, Y., & Ma, X. (2024). To overcome current retention and engagement problems, platforms must begin implementing AI assistance to elevate the learning experience.

### Key trends in the market include:

**The Shift to AI:** There is a growing move toward **AI-powered conversation partners** to bridge the gap between solo study and expensive human tutoring. HelloTalk Blog (2026).

**Acquisition vs. Learning:** Educational research emphasizes subconscious acquisition through meaningful, contextualized input rather than the conscious memorization of grammar rules favored by older apps. EdTech Innovation Hub (2026).



### 1.2. Proposed Solution

The primary aim of The **LinguistAI** is to solve the "fluency plateau" by providing a 24/7, low-pressure conversational environment that mirrors human interaction. The project uniquely addresses the market gaps identified in the competitor review through several core features:

- ❖ **Context-Specific Scenarios:** Users can practice in targeted settings like "Ordering food at a restaurant" or "Asking directions," ensuring the language learned is immediately applicable.
- ❖ **Integrated Multi-Layered Feedback:** Unlike competitors that only evaluate multiple-choice answers, our platform provides real-time feedback on **pronunciation, grammar, and word choice** during a flowing dialogue.
- ❖ **Persistent Conversation Memory:** To ensure realism, the AI companion remembers the context of previous interactions, providing a coherent flow that static chatbots lack.

**LinguistAI** will be a sophisticated web application project that will utilize a robust architecture featuring **Django**, **Wagtail CMS**, and **PostgreSQL** to manage complex user data and persistent conversation memory. Simultaneously, it will draw on data through the integration of **Large Language Models (LLMs)** and sophisticated **Speech-to-Text (STT)** and **Text-to-Speech (TTS)** APIs to facilitate natural, human-like dialogue.

### 1.3. Unique Value Proposition

Unlike conventional competitors that focus on passive memorization or rigid grammatical exercises focused on continuous repetition, **LinguistAI** will transform language acquisition into an active and communicative tool, where the user will be able to interact proactively, through real-time conversations receiving instant feedback.

The main benefit of **LinguistAI** is to bridge the "fluency plateau" by providing a 24/7, pressure-free, AI-driven conversational environment that would be its main feature, this will be reflected in user interaction through specific role-playing and pedagogical feedback in multiple layers, where the big difference will be that users will not only click to memorize the content continuously, but they will actively speak to communicate in specific situations of daily life such as "Order food at a restaurant" or "Directions to order", among many other possibilities, thanks to the fact that the web application will be integrated with an AI that will offer you unlimited options.

#### Key differences from competitors:

**Real-time AI interaction:** Unlike the decontextualized exercises of Duolingo or Babbel, AI Linguist will offer specific and varied scenarios depending on the context that the user wants to practice. In this way, you will be able to practice everyday life situations.

**Pedagogical feedback:** **LinguistAI** will have a real-time phonetic comparison and grammar analysis model, which means it will provide a level of instruction that is normally only present in live tutoring sessions, which typically cost 50 hours per hour.

**Competitive advantage:** **LinguistAI** removes financial barriers to oral practice through a tiered monetization model structured explicitly in Australian Dollars (AUD). The platform offers a completely Free Plan restricted to 5 conversational turns per day to encourage customer acquisition. For serious learners seeking uninterrupted immersion, the **Monthly Premium Plan** is priced at **\$29.99 AUD / month**. Long-term users can opt for the **Annual Premium Plan** billed at **\$239.88 AUD / year** (representing an equivalent rate of \$19.99 AUD / month), delivering a 33% discount. At these

## Project 9: AI Language Learning Companion (IT, Data Science)

rates, users can practice unlimited daily conversations for less than the cost of a single session with an **italki** or **Preply** human tutor, which typically averages \$50 per hour.

The decision to offer a free tier subscription is based on the consumer education technology (EdTech) market, which determines the "reach" price for a subscription to a digital learning web application; This involved assessing consumer price sensitivity, value-based levels, and the critical distinction between monthly and annual billing cycles. The result was that more than 95% of mainstream consumer bases remain locked within the free tiers. Zhang, C., Meng, Y., & Ma, X. (2024). According to that report, **LinguistAI** users will be entitled to get a free account to try out some features.

## 2. Competitor Analysis: An Overview - Written by Jeison & Win

### 2.1. Rosetta Stone - Written by Win

**Website:** <https://au.rosettastone.com/>

**Overview:** Rosetta Stone is one of the most established names in language learning, with over 30 years in the market. Rather than relying on direct translation, the platform employs an immersive, AI-powered methodology that mirrors the natural language acquisition process, encouraging learners to think directly in the target language. Its proprietary speech recognition technology and structured curriculum are designed to serve a broad user base, including individuals, educational institutions, and enterprise clients (Cristiano, 2025).

#### Strengths:

- ❖ Offers courses across 25 languages through a structured, immersive learning method that is particularly effective for beginner and intermediate learners (Cristiano, 2025)
- ❖ TruAccent speech recognition technology provides real-time pronunciation coaching benchmarked against native speakers, a capability that is uncommon at a comparable price point (Cristiano, 2025)
- ❖ Comprehensive learning suite includes live sessions with native speakers, short stories, and learning games, extending beyond a standard language course (Cristiano, 2025)
- ❖ Available across web, iOS, and Android platforms, with offline access supporting consistent daily practice.

#### Weaknesses:

- ❖ Premium pricing relative to competing platforms such as Duolingo may present a barrier for cost-conscious users (Cristiano, 2025)
- ❖ Limited explicit grammar instruction may impede learners who rely on structured grammatical explanation to build language competency (Cooljugator, 2025).
- ❖ Curriculum depth is better suited to beginner and intermediate learners, offering comparatively less value at advanced proficiency levels (Cristiano, 2025)
- ❖ Lacks AI-driven conversational practice features, such as chatbot-style dialogue, which are increasingly available in newer AI-native platforms.



### 2.2. Pimsleur - Written by Win

**Website:** <https://www.pimsleur.com>

**Overview:** Pimsleur is an audio-based language learning platform grounded in decades of linguistic research. Its core methodology, the Pimsleur Method, employs spaced repetition to embed vocabulary into long-term memory, prioritising spoken communication over reading and writing (Cooljugator, 2025). The platform offers courses in 50+ languages and is designed for learners seeking practical conversational ability within a short timeframe (Cooljugator, 2025). Its mobile app supports hands-free learning, making it accessible during commuting or other daily activities, thereby reducing barriers to consistent practice.

#### Strengths:

- ❖ Audio-first methodology is highly effective for developing spoken language skills, with research indicating that learners can achieve conversational ability within a relatively short period of consistent study (Cooljugator, 2025)
- ❖ Broad language catalogue of 50+ languages, including less commonly taught options such as Tagalog, Farsi, and Swahili, catering to diverse linguistic needs (Cooljugator, 2025)
- ❖ Hands-free app functionality supports learning during commutes or other daily routines, accommodating learners with limited time for structured study sessions (Cooljugator, 2025)
- ❖ Voice Coach AI provides real-time pronunciation feedback benchmarked against native speakers, supporting accurate spoken language development (Cooljugator, 2025)

#### Weaknesses:

- ❖ Limited focus on reading, writing, and grammar instruction may not meet the needs of learners requiring comprehensive language proficiency beyond conversational ability (Cooljugator, 2025)
- ❖ Lacks enterprise-tier features such as team management, progress reporting, or business-specific vocabulary modules, limiting its suitability for structured organisational training (Cooljugator, 2025)
- ❖ The predominantly audio format may become repetitive over extended use, potentially affecting learner engagement and long-term retention (Cooljugator, 2025)

### 2.3. Duolingo - Written by Jeison

**Website:** <https://www.duolingo.com/>

**Overview:** Duolingo is the world's most popular language-learning platform, widely known for pioneering the gamification of education. It uses a freemium model with bite-sized, interactive lessons designed to feel like a video game, complete with points, streaks, and leaderboards. Driven heavily by AI and data analytics, Duolingo personalizes lessons based on performance. It has recently introduced "Duolingo Max," a premium tier incorporating generative AI features like conversational chatbots and contextual error explanations.

## Project 9: AI Language Learning Companion (IT, Data Science)

### Strengths:

- ❖ Highly engaging gamified interface with streaks, rewards, and leagues that effectively drives daily user retention and habit formation.
- ❖ Robust free tier makes language learning accessible to a massive global audience.
- ❖ Offers an extensive catalog of over 40 languages, including endangered, constructed (Conlangs), and less commonly taught languages.
- ❖ Recently integrated advanced AI features (Duolingo Max) powered by GPT-4, offering roleplay conversational practice and "Explain My Answer" features.

### Weaknesses:

- ❖ Gamification can sometimes lead users to prioritize maintaining "streaks" and earning points over deep, functional language retention.
- ❖ Translating isolated sentences often lacks the contextual immersion required for real-world fluency.
- ❖ Heavy reliance on text and translation exercises means users may struggle with spontaneous, real-time speech and listening comprehension.
- ❖ Advanced AI features and conversational roleplay are locked behind the most expensive subscription tier and are currently limited to select languages.

## 2.4. Babbel - Written by Jeison

**Website:** <https://www.babbel.com/>

**Overview:** Babbel is a premium, subscription-based platform focused on helping learners achieve real-world, practical conversational skills. Unlike fully algorithmic apps, Babbel's curriculum is empirical, built by a team of over 150 linguists and language teachers. Lessons are tailored specifically to the user's native language (e.g., teaching Spanish differently to an English speaker than to a German speaker). It combines structured grammar instruction, vocabulary review using spaced repetition, and realistic dialogue scenarios, alongside premium live online classes (Babbel Live).

### Strengths:

- ❖ Content is engineered by expert linguists rather than purely generated by algorithms, ensuring high-quality, culturally relevant instruction.
- ❖ Strong focus on practical, real-life dialogue, grammar rules, and vocabulary that can be applied immediately in conversation.
- ❖ Language-pairing methodology utilizes the learner's native tongue to explain grammatical concepts more efficiently.
- ❖ Offers "Babbel Live," allowing users to supplement app learning with small-group, live tutoring sessions led by certified teachers.

### Weaknesses:

- ❖ Offers a more limited catalog of 14 languages, focusing primarily on major European languages.
- ❖ Interface is more traditional and academic, which may feel less engaging or motivating to users who prefer gamified learning.
- ❖ The free version is extremely limited, acting essentially as a short trial before requiring a paid subscription.

## Project 9: AI Language Learning Companion (IT, Data Science)

- ❖ Speech recognition capabilities, while functional for vocabulary check-ins, lack the highly advanced, nuanced real-time feedback found in dedicated audio or AI-native apps.

### 2.5. Summary Table: Competitor Comparison - Written by Jeison

| Feature / Competitor                | Duolingo                                                                    | Babbel                                                                     | Rosetta Stone                                           | Pimsleur                                                    |
|-------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------|---------------------------------------------------------|-------------------------------------------------------------|
| 1. Gamification & Loops             | High (Streaks & leagues)                                                    | Low (Traditional text/audio)                                               | Low (Linear exercises)                                  | Low (Repetitive audio)                                      |
| 2. Conversational AI                | High (Only in 'Max' tier)                                                   | Low (Pre-scripted scripts)                                                 | None (No chatbot)                                       | Medium (Basic voice coach)                                  |
| 3. Hands-Free Study                 | Low (Requires active tapping)                                               | Medium (Standard downloads)                                                | Medium (Standard downloads)                             | High (Audio-first; great for commutes)                      |
| 4. Live Human Coaching              | None (App only)                                                             | High (Babbel Live classes)                                                 | Medium (Add-on studio sessions)                         | None (Pre-recorded only)                                    |
| 5. Native Explanations              | Low (Implicit text parsing)                                                 | High (Tailored grammar rules)                                              | None (100% direct immersion)                            | Low (Focuses on auditory patterns)                          |
| 6. Estimated Base Price (AUD / USD) | Free tier available<br>• Super: ~\$16.99 AUD/mo<br>• Max (AI): ~\$30 AUD/mo | • Monthly: ~\$18 USD/mo<br>• Annual: ~\$9 USD/mo<br>• Lifetime: ~\$300 USD | • Annual: ~\$159 USD/yr<br>• Lifetime: ~\$199–\$399 USD | • Premium: ~\$19.95 USD/mo<br>• All Access: ~\$20.95 USD/mo |

Table 1 - Competitor Comparison.

### 2.6. Key Insights - Written by Jeison

An analysis of the current participants in the market demonstrates that even though the current instruments do work well in aiding the formation of the very first habits and vocabulary, none of them helps the user evolve into a proficient communicator. This brings us to the need for a product that:

**Is Affordable and Interactive:** Provides tiered access, starting from a no-cost baseline up to \$29.99 per month for unlimited 24/7 conversational practice. This structure remains significantly cheaper than a traditional personal tutor who charges between \$15 and \$50 per individual hour.

**Is Educationally Wise:** Can provide feedback concerning pronunciation, grammar usage, and word selection in conversation, unlike the existing gamified software that is focused on providing answers to multiple-choice questions.

**Is Growth-Oriented (Against Gamification):** Concentrated on subconscious learning of language and achieving results rather than gaming the system and maintaining a streak that masks the stagnation in language mastery.

**Is Loaded with Features via the Internet:** Features a robust and performant web application that does not suffer from limitations of a mobile app and can't be made as simplistic as the latter can be.

This gap between "having language skills on paper" and "using language skills in motion" is precisely what LinguistAI intends to bridge.

## 3. The Gap the Project Fills - Written by Jeison

**LinguistAI** is an AI-based solution that addresses various gaps in the current market for learning languages, especially the "fluency plateau" which occurs when learners fail to advance from understanding to speaking fluently.

The gaps that need to be filled include the following:

## Project 9: AI Language Learning Companion (IT, Data Science)

**1. Bridging the "Mute Language" Phenomenon:** This "mute language" is created because popular apps tend to look at language through the prism of mathematics and logic puzzles that have to be solved. This project aims to fill this gap with an artificial intelligence-driven conversation partner, which will emulate the "unpredictability" of human conversation.

**2. Providing Integrated, Multi-Layered Feedback:** Mainstream apps cannot assess free speech. Whereas they can grade a multiple-choice question, they are unable to deliver any feedback regarding the users' sentences and pronunciation. **LinguistAI** is equipped with an artificial intelligence feedback engine, which assesses pronunciation by using a phonetic model and identifies problem words. In addition, it can give instantaneous feedback regarding grammar and vocabulary while engaging in dialogue.

**3. Overcoming the "Gamification Trap":** Popular applications such as Duolingo make use of concepts like streaks and leader boards, which often obscure any real gains from the activity. Students may be concerned more about beating the game than learning the material. With this project, real-life accomplishments and interaction will take priority over extrinsic motivators, and memory functions will allow for persistent dialogue throughout the exchange.

**4. Resolving the Cost and Anxiety of Human Interaction:** Current methods for speaking practice are often hindered by substantial challenges:

- ❖ **Tutor Marketplaces (italki, Preply):** They give great feedback, but are costly (50 per session), and may induce anxiety in learners.
- ❖ **Language Exchanges (HelloTalk):** They are free, yet unreliable since they depend on finding partners.

**LinguistAI** will give learners easy access to frequent speaking practice without spending much money on a tutor or feeling pressured to engage with an actual person.

## 4. Project Aim and Objectives - Written by Jeison

The key objective of the **LinguistAI** is to create the connection between passive learning and active speaking through a less intimidating and round-the-clock conversational atmosphere. The initiative seeks to take learners out of being "mute languages" where learners recognize written languages but fail to speak by applying an AI system to emulate real-life situations.

To achieve this aim, the following **SMART** (Specific, Measurable, Achievable, Relevant, and Time-bound) objectives have been defined:

**Platform Development:** Build an interactive platform with the aid of Django, capable of supporting users' accounts for monitoring their personal progress in several languages.

**Scenario Development:** Develop a list of at least five context-specific conversations (for example, "Ordering food in a restaurant" or "Asking for directions"), differing in level of difficulty, ranging from beginner (A1) to advanced (C2).

**Integration of APIs:** Efficiently incorporate an LLM to generate natural dialogues, STT API to transcribe spoken words in real time, and TTS API to give a voice to the AI characters.

## Project 9: AI Language Learning Companion (IT, Data Science)

**Feedback System:** Design an intelligent feedback system that identifies grammatical mistakes in transcribed text and implements a phonetic comparison algorithm to evaluate the accuracy of pronunciation with regard to particular words.

**Cognitive Continuity:** Ensure cognitive continuity by implementing conversation memory in the AI system for a realistic and coherent conversation flow that accounts for prior conversations.

**Seamless Interaction:** Utilize HTMX to make sure that every interaction with the conversational system is seamless through live injections of responses.

## 5. Project Scope - Written by Jeison

The project scope defines the boundaries of the application, ensuring that the development team focuses on core functionalities that align with the project objectives:

### 5.1. In Scope - Written by Jeison

The following list details the core features and functionalities that **LinguistAI** will deliver as part of its primary system:

**User Management System:** Introduction of secure user accounts so that users can register, login, and monitor their performance in the selected languages.

**Conversation Scenarios:** Pre-selected collection of conversational scenarios such as “Ordering Food at the Restaurant,” “Requesting Directions,” or “Airport” that serve as the context for conversation.

**Adjustable Difficulty Level:** A mechanism whereby learners are enabled to set their skill level from **Beginner (A1)** up to **Advanced (C2)** level and then adjust the AI’s choice of vocabulary and sentence structures in accordance with this skill level.

**Contextual Image Integration:** The lessons would involve the use of an appropriate image, which is essential to help create an immersive learning experience.

#### Integration of Speech Processing Technologies:

- ❖ **Speech-to-text (STT):** Integration of an API to convert the user’s speech to text in real-time.
- ❖ **Text-to-speech (TTS):** Integration of an API to give the AI-powered characters voices.

**LLM-Powered Dialogue Generation:** Application of a sophisticated Large Language Model (LLM) to create interactive answers relevant to the selected scenario, rather than resorting to inflexible pre-scripted dialogues.

**Persistent Conversation History:** Implementation of memory functionality to guarantee that the AI system retains memory of previous exchanges in a conversation.

**Grammar Analysis:** Provides real-time analysis of any grammatical mistakes in the transcribed text from the user.

**Phonetic Pronunciation Assessment:** A module that makes use of a phonetic similarity assessment tool to rate how well the user is pronouncing the words.



## Project 9: AI Language Learning Companion (IT, Data Science)

**Interactive User Interface:** Front-end application built using HTMX to enable smooth, non-blocking interface changes in the chat environment.

### 5.2. Out of Scope - Written by Win

The following will not be implemented in this phase of the project:

**Mobile apps** - LinguistAI is web-only, native iOS or Android apps are not included.

**Live human tutoring** - all conversation partners are AI-driven, no real tutors involved.

**Payment processing** - subscription billing and payment gateway integration will not be built at this stage despite the freemium model being proposed.

**Multi-language UI** - While the backend supports multiple target languages for practice, the user interface (menus, buttons, navigation) remains in English only for this release, no localization support.

**Social features** - leaderboards, friend systems, and peer interaction are not part of this release.

## 6. Functional Requirements (FRs) - Written by Win & Quoc Anh

This section outlines what the system needs to do in order to deliver a functional language learning experience. The requirements are organized across three layers — frontend, server-side, and database — each mapped to a specific technology in the stack PostgreSQL Global Development Group. (2024b). PostgreSQL Global Development Group. (2024c). Together, they describe how a user's voice travels from the browser, through the backend, and back as an AI-spoken response PostgreSQL Global Development Group. (2024a). Mozilla Developer Network (MDN). **Table 2**

### 6.1. Frontend Functionality (UI) - Written by Win

| ID    | Function                                            | Requirement description                                                                                                                                                                                                                                                                         |
|-------|-----------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | User authentication<br>(Bootstrap formsDjango auth) | Users register and log in via an email/password form built with Bootstrap. Client-side validation checks field formats before submission. On success, Django issues a session cookie; failed attempts display inline error messages without a page reload.                                      |
| FR-02 | Scenario selection<br>HTMXBootstrap cards           | A scenario library page displays real-world practice contexts (e.g. "Ordering food", "Asking for directions", "Job interview") as Bootstrap cards grouped by CEFR level (A1–C2). Selecting a scenario sends an HTMX request that loads the chat interface into the page without a full refresh. |
| FR-03 | Voice recording UI<br>(JS MediaRecorderFetch API)   | A record button triggers the browser's MediaRecorder API to capture microphone input. A live waveform or animated indicator confirms active capture. Pressing stop finalises the recording and dispatches the audio file to the Django backend via Fetch API or HTMX hx-post.                   |



## Project 9: AI Language Learning Companion (IT, Data Science)

|              |                                                             |                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FR-04</b> | Transcript display<br>(HTMX swap)                           | After Whisper transcribes the audio, the returned text is injected into the chat interface via an HTMX HTML fragment swap, displaying the user's words as a chat bubble — no page refresh required.                                                                                                                                                                             |
| <b>FR-05</b> | Grammar feedback display<br>(HTMX fragmentBootstrap alerts) | Claude's grammar correction and a 0–100 grammar score are rendered in an HTMX-injected feedback panel below the user's chat bubble. Corrected phrases are highlighted; incorrect tokens are marked in a contrasting colour using Bootstrap utility classes.                                                                                                                     |
| <b>FR-06</b> | AI voice playback<br>(HTML <audio>HTMX swap)                | The ElevenLabs TTS audio file path is embedded in the HTMX response fragment as an HTML <audio> element. The browser auto-plays the AI character's spoken response; manual replay controls are also rendered for accessibility.                                                                                                                                                 |
| <b>FR-07</b> | AI response bubble<br>(HTMX fragment)                       | The AI's text response is injected as a chat bubble in the conversation thread via HTMX fragment swap, so the user can read and listen simultaneously. The text bubble renders independently of audio generation — the <audio> element is delivered asynchronously via a subsequent HTMX swap after the text bubble is displayed, preventing UI blocking during TTS processing. |
| <b>FR-08</b> | Progress dashboard<br>(BootstrapDjango views)               | A personal dashboard page (server-rendered by Django, styled with Bootstrap) displays session history, average grammar score over time, and a word list flagging terms that have been consistently mispronounced or misused — sourced from the utterances table in PostgreSQL.                                                                                                  |

Table 2 - Frontend Functionality (UI)

### 6.2. Server-Side Logic - Written by Win

| ID           | Function                          | Requirement Description                                                                                                                                                                       |
|--------------|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FR-09</b> | User Validation                   | Django validates each incoming request before processing. Unauthenticated users are blocked from accessing lessons or audio endpoints. Sessions are managed server-side securely.             |
| <b>FR-10</b> | Audio Verification & Handling     | Django receives the uploaded audio file, verifies the format and size, then passes it through pydub/librosa for preprocessing before forwarding to the STT API.                               |
| <b>FR-11</b> | Transcription (STT)               | Django sends the verified audio to OpenAI Whisper API. The returned transcript is validated for completeness and passed to the LLM pipeline.                                                  |
| <b>FR-12</b> | Prompt Assembly & LLM Integration | Django combines the transcript, lesson context, difficulty level, and conversation history into a structured prompt. This is sent to the LLM (Claude/GPT-4) which returns grammar corrections |

## Project 9: AI Language Learning Companion (IT, Data Science)

|       |                        |                                                                                                                                                                                                                |
|-------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       |                        | and a conversational reply. Retries and failures are handled gracefully.                                                                                                                                       |
| FR-13 | Voice Generation (TTS) | Django sends the AI's text response to ElevenLabs API to generate a realistic audio file. The file path is saved to the database and returned to the front-end via an <audio> tag.                             |
| FR-14 | Data Storage           | Django saves audio file paths, transcripts, grammar scores, and pronunciation metrics into PostgreSQL under the user's session ID. Each record is linked to its lesson via foreign key.                        |
| FR-15 | Data Security          | All API keys are stored server-side as environment variables. Data is encrypted in-transit via HTTPS/TLS 1.3. Audio files and transcripts are scoped to the user's session, preventing cross-user data access. |

Table 3 - Server-Side Logic Table

**Supporting Note:** All external API calls (Whisper, LLM, ElevenLabs) are handled exclusively server-side — no API keys are ever exposed to the front-end.

### 6.3. Database Layer (PostgreSQL via Supabase) - Written by Quoc Anh

To ensure data integrity and minimize redundancy, the relational database is designed adhering to the Third Normal Form (3NF) principles (Elmasri & Navathe, 2015).

Furthermore, to securely manage transactional data and prevent Insecure Direct Object Reference (IDOR) vulnerabilities, UUID data types are utilized as primary keys instead of auto-incrementing integers (PostgreSQL Global Development Group, 2024b).

For handling the dynamic and complex AI metrics generated by the LLM, the **interaction\_logs** table leverages PostgreSQL's advanced JSONB column, allowing for scalable and flexible data storage without requiring frequent schema alterations (PostgreSQL Global Development Group, 2024a).

#### 6.3.1. Tables and fields

- 1) **users** — manage user accounts, authentication data, and dynamic proficiency levels to allow AI personalization

| Column Name     | Data Type    | Constraints                       | Description                                                             |
|-----------------|--------------|-----------------------------------|-------------------------------------------------------------------------|
| id              | UUID         | Primary Key                       | Unique identifier for the user.                                         |
| username        | VARCHAR(50)  | Not Null, Unique                  | User's display name.                                                    |
| password_hash   | VARCHAR(255) | Not Null                          | Securely hashed password (Django default).                              |
| target_language | VARCHAR(20)  | Not Null,<br>Default<br>'English' | The language the user is currently practicing (e.g., 'en', 'ja', 'vi'). |

## Project 9: AI Language Learning Companion (IT, Data Science)

|                          |             |                    |                                                                    |
|--------------------------|-------------|--------------------|--------------------------------------------------------------------|
| <b>proficiency_level</b> | VARCHAR(20) | Default 'Beginner' | Dynamic level (Beginner/Inter/Advanced) used to adjust AI prompts. |
| <b>subscription_plan</b> | VARCHAR(20) | Default 'Free'     | Defines the user's active subscription tier.                       |
| <b>created_at</b>        | TIMESTAMP   | Default NOW()      | Account creation timestamp.                                        |

Table 4 - Users Table

**Supporting Note:** A clear distinction exists between the platform's user interface and its data processing capabilities. As defined in the project scope, the system's operational interface (menus, navigation dashboards, and forms) is strictly English-only. Conversely, the backend database layer—specifically the **target\_language** field in the users table—is engineered to store multi-lingual parameters (e.g., 'en', 'ja', 'vi'). This setup ensures that while the front-end interface remains uniform and in English, the backend can successfully route and process distinct foreign language payloads through the external AI models.

- 2) **scenarios** — Stores predefined learning situations and multimedia assets (video contexts) to support immersive role play.

| Column Name          | Data Type    | Constraints | Description                                           |
|----------------------|--------------|-------------|-------------------------------------------------------|
| <b>id</b>            | INT          | Primary Key | Unique ID for the scenario (Auto-increment).          |
| <b>title</b>         | VARCHAR(100) | Not Null    | Name of the scenario                                  |
| <b>system_prompt</b> | TEXT         | Not Null    | Core prompt defining the AI character's persona.      |
| <b>video_url</b>     | VARCHAR(255) | Nullable    | URL to the background video for dynamic UI rendering. |

Table 5 – Scenarios Table

- 3) **learning\_sessions** — tracks individual practice runs, grouping multiple conversational turns into a single cohesive learning event.

| Column Name          | Data Type    | Constraints   | Description                                         |
|----------------------|--------------|---------------|-----------------------------------------------------|
| <b>id</b>            | UUID         | Primary Key   | Unique identifier for the session.                  |
| <b>user_id</b>       | UUID         | Foreign Key   | Links to users.id                                   |
| <b>scenario_id</b>   | INT          | Foreign Key   | Links to scenarios.id                               |
| <b>started_at</b>    | TIMESTAMP    | Default NOW() | When the session began.                             |
| <b>overall_score</b> | Decimal(5,2) | Nullable      | Aggregate score calculated upon session completion. |

Table 6 - Learning Sessions Table

## Project 9: AI Language Learning Companion (IT, Data Science)

4) **interaction\_logs** — records real-time spoken interactions and complex AI evaluations.

| Column Name              | Data Type    | Constraints   | Description                                                                                              |
|--------------------------|--------------|---------------|----------------------------------------------------------------------------------------------------------|
| <b>id</b>                | UUID         | Primary Key   | Unique ID for this specific conversational turn.                                                         |
| <b>session_id</b>        | UUID         | Foreign Key   | Links to learning_sessions.id                                                                            |
| <b>user_audio_url</b>    | VARCHAR(255) | Not Null      | Cloud storage path to the user's recorded .wav file.                                                     |
| <b>user_transcript</b>   | TEXT         | Not Null      | Whisper STT output of the user's speech.                                                                 |
| <b>ai_audio_url</b>      | VARCHAR(255) | Not Null      | Path to the TTS audio generated by the AI (ElevenLabs)                                                   |
| <b>ai_response_text</b>  | TEXT         | Not Null      | The AI character's conversational reply.                                                                 |
| <b>detailed_feedback</b> | JSONB        | Nullable      | Stores flexible AI metrics(grammar fixes, pronunciation scores, emotion tags) without schema alteration. |
| <b>created_at</b>        | TIMESTAMP    | Default NOW() | Timestamp of this specific interaction.                                                                  |

Table 7 – Interaction Logs Table

### 6.3.2. Entity-Relationship

To guarantee data consistency and enforce the *Third Normal Form (3NF)* across the application, the database schema implements strict entity-relationship mapping using foreign key constraints. This relational design prevents orphaned records and ensures that learning sessions and interaction logs are perfectly synchronized with their respective users (Elmasri & Navathe, 2015)

- ❖ **users (1) to learning\_sessions (N)**: A user can initiate multiple learning sessions, but each session is strictly owned by one user.
- ❖ **scenarios (1) to learning\_sessions (N)**: A specific roleplay scenario can be utilized across many different sessions by various users.
- ❖ **learning\_sessions (1) to interaction\_logs (N)**: A single session consists of multiple conversational turns (logs). Deleting a session will cascade and remove all associated interaction logs to prevent orphaned data.

### 6.3.3. Indexes for Performance Optimization

Addressing the strict non-functional requirements for system performance, B-Tree indexes are applied to highly queried columns such as **user\_id** and **session\_id**. These indexes significantly speed up data retrieval, ensuring the database can handle real-time conversational queries with minimal latency, effectively preventing expensive full-table scans (PostgreSQL Global Development Group, 2024c).

1. **idx\_users\_username** on **users(username)**: Optimizes authentication queries during user login.

## Project 9: AI Language Learning Companion (IT, Data Science)

2. **idx\_sessions\_user\_id** on **learning\_sessions(user\_id)**: Prevents full-table scans when querying a user's dashboard to display their historical progress and past sessions.
3. **idx\_logs\_session\_id** on **interaction\_logs(session\_id)**: Crucial for maintaining conversational memory. It ensures that the backend can instantly fetch previous chat logs within an active session to provide the LLM with immediate context.

### 6.3.4. Row Level Security (RLS) Policies

Security is a paramount non-functional requirement, especially when handling biometric voice data. The system utilizes Supabase's Row Level Security (RLS) policies to enforce the principle of least privilege at the database level. This guarantees that users can only access and modify their own conversational logs and practice sessions, completely isolating personal data across the multi-tenant architecture (Supabase, 2024)

- ❖ **Principle of Least Privilege:** By default, all tables restrict access.
- ❖ **learning\_sessions Policy:** SELECT, INSERT, UPDATE, DELETE operations are only permitted if `auth.uid() == user_id`. A user can only view and modify their own sessions.
- ❖ **interaction\_logs Policy:** Access is granted by joining the **learning\_sessions** table to verify ownership. A user can only fetch logs associated with a session they own.
- ❖ **Scenarios Policy:** Read-only (SELECT) access is granted to all authenticated users, as these are global system templates.

## 6.4. Backend Workflow Overview - Written by Quoc Anh

Much of the heavy lifting in our system happens entirely behind the scenes. While users interact with a seamless interface, the backend acts as the unseen engine powering the interactive platform for language learners. This server-side infrastructure is responsible for securely managing accounts, storing detailed conversational logs, routing audio to various AI APIs, checking real-time answers, and enforcing daily usage limits based on subscription plans. It also ensures that the platform remains stable and responsive even when multiple students are practicing simultaneously.

When a learner first registers, the system executes strict encryption and decryption protocols to securely hash their password before saving it to the database (Elmasri & Navathe, 2015). Once they log in, the backend issues a secure session token. Think of this token as a digital ID card that the server checks continuously to verify which account belongs to which person belongs to. Since we are handling sensitive, biometric voice data, this step is vital—without it, a user might accidentally access someone else's practice session or feedback profile.

Before the conversation begins, the user selects a learning context from a dropdown menu, such as "Ordering food at a restaurant." At this point, the backend immediately performs detailed data handling by verifying the user's subscription tier. For instance, a "Free Plan" might limit the user to five conversations a day, whereas a "Premium Plan" allows for unlimited practice. If authorized, the system retrieves the specific AI system prompts needed to set up the persona for that scenario. When the user holds the microphone and speaks, the front-end securely transmits the audio recording (Mozilla Developer Network, 2024). The backend first conducts rigorous data verification to ensure the incoming file is safe and correctly formatted. It then sends the audio payload straight to the Speech-to-Text API for minimal-latency transcription (OpenAI, 2024). The server merges this transcribed text with the scenario's specific context and pushes it to the Large Language Model (GPT-4) to act as the AI-powered conversation partner.



## Project 9: AI Language Learning Companion (IT, Data Science)

Once the LLM sends back the conversational reply and calculates real-time feedback on pronunciation, grammar, and word choice, the backend reviews the data to ensure accuracy. It then routes the text to the Text-to-Speech API to generate a natural, human-sounding voice response (ElevenLabs, 2024). All of these metrics, transcripts, and audio file paths are then seamlessly integrated and saved securely into a well-organized database table using advanced JSON structures (PostgreSQL Global Development Group, 2024a).

Finally, Django runs all the backend code and seamlessly integrates how the front-end, server-end, and database interact. Instead of forcing a page reload to show the results, the server packages the AI's feedback into an HTML fragment. Using HTMX, this fragment is injected directly into the chat interface (Carson, 2024), ensuring the experience feels exactly like a fluid, spoken conversation with an AI character.

## 7. Non-Functional Requirements (NFRs) - Written by Camilo

These requirements describe how the system should perform, not just what it does. They cover things like speed, security, and ease of use. Each one has a specific, measurable target so we can verify it during development (Norman, D. A., 2013). **Table 8**

**1. Performance:** Performance requirements ensure that the real-time conversation pipeline, speech recording, transcription, AI response and audio playback runs fast enough to feel natural. A delay of more than a few seconds at any step breaks the conversational flow and undermines the app's core purpose (Godwin-Jones, R., 2021).

**2. Reliability:** Because the app depends on three external APIs, Whisper, Claude and ElevenLabs TTS, failures outside the team's control are inevitable. These requirements ensure the system degrades gracefully and users always receive a clear, actionable response rather than a crash or blank screen.

**3. Security:** Users share personal information and voice recordings with this application. As an Australian-based project, the team has legal obligations under the **Privacy Act 1988** (Australian Government — Attorney-General's Department, 2023), specifically APP 3 (collect only what is necessary) and APP 11 (keep personal information secure) (Office of the Australian Information Commissioner, 2023). Security requirements also protect the project from common web vulnerabilities identified in the OWASP Top 10 (OWASP Foundation, 2021).

**4. Usability:** Language learners, the primary users of this app, may not be technically confident. Usability requirements ensure the interface is intuitive enough to be used without instructions, accessible to users with disabilities and functional across common devices (Nielsen, J., 1993).

**5. Scalability:** While the initial deployment is for a small user group, the system must be designed to handle growth without architectural changes. The two key scalability concerns are concurrent users and long-term database performance.



## Project 9: AI Language Learning Companion (IT, Data Science)

| ID            | Category | What it means                                                                                                                                                                                                                                                                                                   | Target                                                                                                            | How it will be checked                                                       |
|---------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>NFR-01</b> | 1        | After the user stops recording, their spoken words must appear as text on screen within 3 seconds. This covers the full process: the audio is sent to Whisper for transcription, then the result is displayed in the chat (Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I., 2023). | Transcript appears in under 3 seconds for any recording up to 30 seconds long.                                    | Timed during integration testing using browser DevTools.                     |
| <b>NFR-02</b> | 1        | Once the transcript is ready, Claude must start sending back a response within 5 seconds and the full reply must finish within 15 seconds. The response streams word by word so the user doesn't stare at a blank screen (Anthropic, 2024).                                                                     | First word appears in under 5 seconds. Full reply done in under 15 seconds.                                       | Timed across 50 test messages during integration testing.                    |
| <b>NFR-03</b> | 1        | After Claude finishes writing its response, the AI's spoken audio (generated by ElevenLabs) must start playing within 2 seconds. The audio loads at the same time as the text bubble.                                                                                                                           | Audio starts playing in under 2 seconds after the AI reply finishes.                                              | Measured from HTMX response to browser audio play event.                     |
| <b>NFR-04</b> | 1        | Any time the page updates, like showing a new chat bubble or loading the feedback panel, it should feel instant. Since we use HTMX instead of full page reloads, these updates must be completed quickly (BigSky Software, 2024).                                                                               | All page updates complete in under 1 second.                                                                      | Checked using Chrome DevTools during a test session.                         |
| <b>NFR-05</b> | 2        | The app needs to be available whenever students want to practice. Unexpected downtime frustrates users and breaks learning routines.                                                                                                                                                                            | The app is available at least 99% of the time during study hours (8am–10pm AEST, weekdays).                       | Monitored with UptimeRobot after deployment.                                 |
| <b>NFR-06</b> | 2        | If Whisper or Claude stops responding, the user should always see a clear message and have an option to continue. The app must never crash or show a blank Django error page.                                                                                                                                   | Any API failure shows a friendly error message within 10 seconds. A text input fallback appears if Whisper fails. | Each API temporarily disabled during testing to confirm correct fallback.    |
| <b>NFR-07</b> | 2        | User progress, grammar scores and session history are stored in the database. If something goes wrong, we need to be able to recover that data without losing everything.                                                                                                                                       | Database backed up every 24 hours. Can be restored in under 1 hour if needed (PostgreSQL Global                   | Backup tested and restored procedure documented in the README before launch. |

## Project 9: AI Language Learning Companion (IT, Data Science)

|               |   |                                                                                                                                                                                                                                                    |                                                                                                                                                     |                                                                                                    |
|---------------|---|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
|               |   |                                                                                                                                                                                                                                                    | Development Group, 2024).                                                                                                                           |                                                                                                    |
| <b>NFR-08</b> | 3 | Everything the user sends to the server, including voice recordings and login details, must be encrypted so it can't be read by anyone in between.                                                                                                 | All traffic uses HTTPS with TLS 1.3. Qualys SSL Labs score must be A or above (OWASP Foundation, 2021).                                             | SSL Labs test run on the production domain before submission.                                      |
| <b>NFR-09</b> | 3 | User passwords must never be stored as plain text. Login sessions must be protected so other scripts can't access them. Too many failed login attempts must be blocked.                                                                            | Passwords hashed with Django's PBKDF2. Secure session cookies. Max of 5 failed logins per minute per IP (Django Software Foundation, 2024).         | Database inspected for plaintext passwords. Cookie flags verified in browser DevTools.             |
| <b>NFR-10</b> | 3 | Voice recordings are private. Once Whisper has turned the audio into text, the audio file must be deleted. We only keep the text transcript, not the recording itself.                                                                             | Audio file deleted within 30 seconds of transcription completing. Only the text is saved (Office of the Australian Information Commissioner, 2023). | Automated test checks upload folder is empty within 30 seconds of each request.                    |
| <b>NFR-11</b> | 3 | Our API keys for Whisper, Claude and OpenAI TTS must never appear in the code that runs in the browser or in our GitHub repository (Django Software Foundation, 2024).                                                                             | API keys stored as environment variables on the server only. Zero keys exposed in the frontend or Git history.                                      | Repository scanned with truffleHog. Browser network requests inspected to confirm no keys visible. |
| <b>NFR-12</b> | 4 | Language learners are our main users, not developers. The app must be simple enough that a brand-new user can sign up and finish their first conversation without reading any instructions.                                                        | At least 4 out of 5 first-time users complete their first conversation in under 5 minutes without help (Nielsen, J., 1993).                         | Think-aloud usability test with 5 participants before the oral presentation.                       |
| <b>NFR-13</b> | 4 | The app must be accessible to users with disabilities (W3C, 2018). This is also required by Australian law (Australian Government, 1992). Bootstrap 5 gives us a solid accessible baseline, but we still need to verify it (Bootstrap Team, 2024). | Meets WCAG 2.1 Level AA. Zero accessibility errors on all main screens.                                                                             | axe-core browser extension scan on each screen. Full keyboard navigation test.                     |

## Project 9: AI Language Learning Companion (IT, Data Science)

|               |   |                                                                                                                                       |                                                                                                                                                                                 |                                                                                               |
|---------------|---|---------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>NFR-14</b> | 4 | Students use different devices to study. The app must work properly on desktop and tablet screens and on the most common browsers.    | Works on Chrome, Firefox, and Edge (latest versions) and on screens 768px wide or larger.                                                                                       | Manual testing on two browsers and a tablet-sized viewport before submission.                 |
| <b>NFR-15</b> | 5 | A whole class might use the app at the same time. The server must handle multiple users without slowing down noticeably.              | 50 users at the same time, each having one conversation turn every 30 seconds, without response times getting more than 50% slower (PostgreSQL Global Development Group, 2024). | Locust load test before the system is opened to real users.                                   |
| <b>NFR-16</b> | 5 | Over time, the database will grow as more users' complete sessions. The progress dashboard must stay fast even with a lot of records. | Progress dashboard loads in under 2 seconds even with 10,000 session records in the database (PostgreSQL Global Development Group, 2024).                                       | Test dataset of 10,000 records inserted. Dashboard load time confirmed with browser DevTools. |

Table 8 - Non-Functional Requirements Table

## 8. Cost and Scalability - Written by Jeison

A conversational pipeline that uses Django, HTMX, and Supabase in real-time eliminates the need for typical app store charges but introduces variable platform operational expenses. The sole reason for this is the cost of artificial intelligence processing APIs. (Carson, 2024; Supabase, 2024).

### 8.1. API Consumption Costs (AUD)

Three significant steps are involved in an individual "conversational turn": Speech-to-Text (STT) through OpenAI Whisper, and Anthropic Claude 3.5 to use Large Language Model (LLM) contextual reasoning. Synthetic voice generation using ElevenLabs Text-to-Speech (TTS) and Sonnet. (Anthropic, 2024; ElevenLabs, 2024; OpenAI, 2024). The structural baseline dependencies in Australian currency are outlined in **Table 9**.

## Project 9: AI Language Learning Companion (IT, Data Science)

| Service Tier Dependency    | Underlying Baseline Metric     | Unit Rate (AUD)     | Cost Per Conversational Turn |
|----------------------------|--------------------------------|---------------------|------------------------------|
| OpenAI Whisper (STT)       | Per minute of user speech      | \$0.0085 / min      | \$0.0021                     |
| Claude 3.5 Sonnet (Input)  | Per 1 Million context tokens   | \$4.25 / M tokens   | \$0.0013                     |
| Claude 3.5 Sonnet (Output) | Per 1 Million generated tokens | \$21.25 / M tokens  | \$0.0021                     |
| ElevenLabs API (TTS)       | Per 1,000 text characters      | \$0.21 / K chars    | \$0.0210                     |
| Supabase Data Layer        | Storage/Query transaction logs | Database allocation | \$0.0003                     |
| Total Base Cost            | Per Conversational Turn        | —                   | \$0.0268                     |

Table 9 - Unit Cost Matrix for AI API Infrastructure (AUD)

### 8.2. Operational Subscription Scaling Tiers

Assuming a variable charge of **\$0.0268 per conversational turn**, the running expenses will vary for each user subscription level:

- ❖ **Free Account Plan:** A user is only allowed 5 conversational turns per day (\$150 turns/month). This limits the maximum cost to the platform to \$4.02 per user monthly, which is a kind of marketing customer acquisition cost (CAC) directly covered by the premium subscription plan limits.
- ❖ **Premium Subscription Tier:** Active intermediate learners are considered as generating 30 turns daily on average (\$900 turns/month). Therefore, the base cloud operational cost is expected to be \$24.12 per user monthly.

### 8.3. Architectural Performance & Processing Bottlenecks

In order to keep conversations natural, real-time dialogue processing needs running through complete exchanges in less than 3 seconds. (Godwin-Jones, 2021).

- ❖ **Database Scaling:** For interaction metrics storage, **PostgreSQL JSONB** field is being used which allows making immediate changes without the requirement of altering the main database schema. B-Tree indexes are also being used on **user\_id** and **session\_id** keys for ensuring that the history fetching remains below 2 seconds even if the logs have more than 10, 000 dashboard rows. (PostgreSQL Global Development Group, 2024).
- ❖ **Handling Latency Blockages:** Since the sequential tracking of through Whisper, Claude, and ElevenLabs leads to server latency issues, an approach has been found that makes use of server workers along with front-end HTMX fragment swaps so that the text updates appear on the chat screen even before the audio payload associated with it finishes loading. This allows an uninterrupted interaction pipeline.

## 9. Cost-Benefit and Competitive Analysis - Written by Jeison

### 9.1. Financial and Pedagogical Comparison

**LinguistAI** occupies an open space between low-cost, passive gamification apps and expensive human tutor marketplaces (HelloTalk, 2026). **Table 10** evaluates options through a direct financial lens.

| Evaluation Factor       | Traditional Apps (e.g., Duolingo) | Marketplace Tutors (e.g., iTalki) | LinguistAI Platform                       |
|-------------------------|-----------------------------------|-----------------------------------|-------------------------------------------|
| Average Monthly Cost    | \$0.00 to \$21.00 / month         | \$210.00 to \$560.00 / month      | \$0.00 (Free) / \$29.99 (Premium) / month |
| Active Practice Hours   | Infinite text selection loops     | Limited by booked lesson hours    | Unlimited 24/7 vocal interactions         |
| Learner Vulnerability   | Low; detached mechanics           | High; conversational anxiety      | Zero social pressure or judgment          |
| Correction Metric Depth | Binary right/wrong patterns       | Variable; human-dependent reviews | Immediate pronunciation/grammar checks    |

Table 10 - Comparative Analysis of Language Learning Frameworks.

### 9.2. Market Positioning Strategy (Bypassing the Fluency Plateau)

The platform's core value proposition addresses the **Fluency Plateau**, a common baseline barrier where learners master passive grammar rules but struggle to speak spontaneously (EdTech Innovation Hub, 2026). **Figure 1**

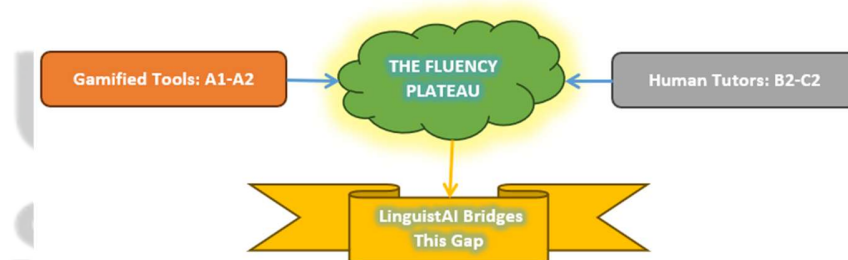


Figure 1 - Bypassing the Fluency Plateau

**Addressing the issue of language learning without speaking:** Classic language learning software's usually have translation exercises that are rigid ways of logic solving puzzles, that often make it impossible for learners to come up with the speech on their own in the real-life situation (Maggi, 2025). **LinguistAI** solves this problem through roleplay sessions that change unpredictably and mimic real-life communication.

**Innovative layered feedback system:** Many language learning applications are limited to multiple-choice type questions and do not have the features on their mobile platforms that are suitable for recording real-time speech (Languages Around the Globe, 2026). **LinguistAI** provides immediate feedback by simultaneously registering phonetic changes and grammatical errors.



## Project 9: AI Language Learning Companion (IT, Data Science)

**Breaking down financial barriers:** Although tutors hired from the human marketplace can give very detailed feedback, their hourly rates are very high (**\$50-\$70+ per hour**), thus, it is not feasible to practice speaking regularly. **LinguistAI** provides continued conversational immersion at a fraction of the cost of one human session, thus alleviating the user's speaking anxiety and increasing their desire to communicate (Zhang et al., 2024).

### 9.3. Return on Investment (ROI) Analysis

Assessing the financial feasibility of LinguistAI involves weighing upfront engineering costs against ongoing subscriptions in Australia's education technology market. Hosting the app on the web with Django, HTMX, and Supabase means no 30% revenue share of the marketplace (Carson, 2024; Supabase, 2024). This hardware structure is aimed at getting the highest possible net profit for being able to have long generative AI processing queues.

#### 9.3.1. Financial Modeling Parameters (AUD)

- ❖ **Fixed CapEx (Year 1):** \$45,000 AUD total payment (includes UI/UX design, software development, security checking, domain set-up).
- ❖ **Variable OpEx Tiers:** Based on a core processing fee of \$0.0268 AUD per conversational turn (Anthropic, 2024; OpenAI, 2024). Free accounts have a monthly cost of \$4.02 AUD (capped); Premium active accounts may spend \$24.12 on average per month.
- ❖ **Subscription Revenue Model:** Premium plans are sold for \$29.99 AUD per month, or students can also pay \$239.88 AUD for a yearly subscription.

#### 9.3.2. Projections and Break-Even Point (12-Month)

**Table 11** maps the platform's financial milestones over its initial year of production.

| Operational Metric                   | Quarter 1 | Quarter 2 | Quarter 3 | Quarter 4 |
|--------------------------------------|-----------|-----------|-----------|-----------|
| Total Registered Users               | 1,200     | 3,500     | 7,000     | 12,000    |
| Paid Premium Subscribers (5%)        | 60        | 175       | 350       | 600       |
| Gross Quarterly Revenue (AUD)        | \$5,398   | \$15,744  | \$31,489  | \$53,982  |
| Total Cloud Processing OpEx (AUD)    | \$4,212   | \$12,285  | \$24,570  | \$42,120  |
| Quarterly Net Operating Margin (AUD) | \$1,186   | \$3,459   | \$6,919   | \$11,862  |
| Cumulative Operating Profit (AUD)    | \$1,186   | \$4,645   | \$11,564  | \$23,426  |

Table 11 -12-Month Scaled Revenue and Cost Projections (AUD)

$$\text{Break-Even Point} = \frac{\text{Fixed CapEx}}{\text{Average Monthly Premium Net Margin Per User}}$$

## Project 9: AI Language Learning Companion (IT, Data Science)

One premium user will provide the net operating profit of \$5.87 AUD monthly (subscription fee is \$29.99 while the unit token cost is \$24.12). As one premium subscriber supports the platform usage of 1.4 free users, the project comes to breaking even financially at Month 10 with 520 active premium users simultaneously.

### 9.3.3. Profitability and Financial Viability

**LinguistAI** is proving to have a highly promising commercial potential and self-sufficient profitability in the Australian EdTech sector. Being a decoupled web app, it uses **Django**, **HTMX**, and **Supabase**, which entirely removes the 30% marketplace distribution fee, thus enabling internal operational margins to cover the background generative AI processing pipelines.

This platform's economic viability is supported by its excellent unit economics: a monthly Premium subscription at **\$29.99 AUD** yields a gross operational margin of **\$5.87 AUD after the deduction of the cloud API cost of \$24.12 AUD**. Not only does this structural margin well cover the cost of free-user support (max \$4.02 AUD monthly), but even with a modest 5% conversion rate of 12, 000 registered users, the venture pays off its **\$45, 000 AUD fixed CapEx** completely by **Month 10**, having an expected **\$23, 426 AUD** in cumulative net profit by Year 1.

## 10. Proposed Pricing Plans - Written by Jeison

In order to stay financially sound and at the same time reduce the entry barriers to market, LinguistAI implements a multi-level subscription system whose price is exclusively set in Australian Dollars (AUD). The fact that it is a web-based service designed using Django and HTMX allows it to avoid paying the 30% commission to digital distribution through mobile store (Carson, 2024). Thanks to this structural decision, the company is able to channel its margin into the development of high-speed AI generative inspiration pipelines running in the background.

### 10.1. Subscription Tier Structure

The framework categorizes users into three separate subscription plans based on the extent of their backend daily processing requests, effectively managing user traffic versus essential cloud API expenses.

- ❖ **Free Plan (Basic Access):** Users are limited to 5 conversational turns per day (\$150 turns/month). This level is targeted at beginners, and they get the chance to interact with real-time speech without having to pay upfront. Essentially, it acts as a customer acquisition channel where the maximum variable cloud cost that a user can generate is \$4.02 AUD per month and is covered by the paid tiers.
- ❖ **Monthly Premium Plan (\$29.99 AUD / month):** This plan is tailored for serious language learners. It offers unlimited conversational turns and full access to all situational practice environments. It supports an average intermediate user who performs 30 conversational turns per day (\$900 turns/month) and results in an individual monthly cost of \$24.12 AUD.
- ❖ **Annual Premium Plan (\$239.88 AUD / year Billed Annually):** Reduces the starting price to the monthly equivalent of \$19.99 AUD, giving long-standing customers a 33% discount. With

## Project 9: AI Language Learning Companion (IT, Data Science)

this plan, the company obtains the funds for the initial operation that helps them with stabilizing the costs of development and covering infrastructure during peak times.

### 10.2. Competitive Pricing Alignment

**LinguistAI** balances cost and educational value, positioning itself strategically alongside established global platforms within the Australian market. **Table 12** outlines this competitive landscape.

| Platform Strategy   | Monthly Price (AUD)  | Yearly Plan Price (AUD) | Conversational Output Model    | Comprehensive Feedback System |
|---------------------|----------------------|-------------------------|--------------------------------|-------------------------------|
| Duolingo Super      | \$18.99 / month      | \$129.96 / year         | Restricted translation prompts | Simple pattern matching       |
| Duolingo Max        | \$30.00 / month      | \$168.00 / year         | Scripted AI interactions       | Short context reviews         |
| Babbel Premium      | \$17.99 / month      | \$149.00 / lifetime     | Fixed sentence repetition      | Basic voice recognition       |
| iTalki Marketplace  | \$210.00 to \$560.00 | —                       | Live human conversation        | Unstructured tutor reviews    |
| LinguistAI Platform | \$29.99 / month      | \$239.88 / year         | Adaptive 24/7 roleplay         | Phonetic & grammar feedback   |

Table 12 - Competitive Subscription and Feature Matrix (AUD)

### 10.3. Market Monetization Logic

Traditional apps like Duolingo Super operate on a very fixed and simple model of fill-in-the-blank that most of the time makes the learner recognize text only and they never develop their ability to talk (HelloTalk, 2026). Duolingo Max at its best introduces a very basic and limited interaction with AI, which is mostly short interactions and fully controlled by the script without detailed pronunciation scoring (Copycat Cafe, 2026).

On the other hand, hiring professional tutors in marketplace networks like **iTalki** offers conversational immersion but also creates financial obstacles for majority of students as they have to pay hundreds of dollars a month for just a few hours of practice.

**LinguistAI** basically addresses the problem of inaccessible speaking skills development by giving unlimited access to talking sessions for a fixed fee of **\$29.99 AUD per month**. The subscription model of paying a monthly charge keeps the program running and enables it to provide a highly effective speaking tool in an affordable way for the learners in general while balancing the **\$0.0268 AUD cost per conversational turn**.

## 11. Use Case - Written by Camilo & Quoc

The following use case diagrams define the functional scope of **LinguistAI** by identifying the system's actors, use cases and the relationships between them. The diagrams are organized into three sections: Authentication & Configuration, which covers the setup layer before a session begins; Conversation Core, which captures the real-time voice and AI pipeline and Persistence & Progress, which handles data storage and learner progress tracking. Together, they provide a complete picture

## Project 9: AI Language Learning Companion (IT, Data Science)

of how the system behaves from the user's perspective and how each component interacts to deliver the language learning experience.

**Use Case Diagram — LinguistAI**

| Actor                        | Type      | Description                                                                                              |
|------------------------------|-----------|----------------------------------------------------------------------------------------------------------|
| <b>Learner</b>               | Primary   | The end user who registers, selects scenarios, speaks, and receives feedback from the system             |
| <b>OpenAI Whisper (STT)</b>  | Secondary | External API that transcribes the user's recorded audio into text in real time                           |
| <b>LLM (Claude / GPT-4)</b>  | Secondary | External large language model that generates conversational replies and grammar corrections              |
| <b>ElevenLabs (TTS API)</b>  | Secondary | External API that converts the AI's text reply into natural, human-sounding audio                        |
| <b>Supabase / PostgreSQL</b> | Secondary | Persistent database layer that stores users, sessions, interaction logs, and enforces Row Level Security |

*Table 13 - Use Case Diagram — LinguistAI*

### 11.1. Register / Login (UC1)

Allow a new user to create an account or an existing user to log in. Django handles credential validation and session management. This is the entry point to the system; no other use case is accessible without completing this step.

**Primary Actor:** Learner

**Secondary Actor:** None

**Preconditions:**

- ❖ User has internet access
- ❖ Email is not already registered (for sign-up)

**Main Flow:**

1. Learner opens LinguistAI and selects "Sign Up" or "Login"
2. System displays the Bootstrap email/password form
3. Client-side validation checks field formats before submission
4. Django authenticates credentials via django.contrib.auth
5. System issues a secure session cookie
6. Learner is redirected to the scenario selector page

## Project 9: AI Language Learning Companion (IT, Data Science)

### Alternative Flow:

- ❖ 3a. Invalid format → inline field error displayed, no page reloads
- ❖ 4a. Wrong credentials → "Invalid email or password" banner shown

### Postconditions:

- ❖ Active session cookie present in the browser
- ❖ Learner can access all authenticated endpoints

## 11.2. Select Scenario (UC2)

Allows the Learner to browse and select a conversation scenario from a categorized library. Each scenario defines the context and AI persona for the upcoming practice session. Selecting a scenario initializes a new learning session in the database.

**Primary Actor:** Learner

**Secondary Actor:** None

### Preconditions:

- ❖ UC1 completed (active session)

### Main Flow:

1. System displays the scenario library as Bootstrap cards grouped by CEFR level (A1–C2)
2. Learner clicks a scenario card (e.g., "Ordering food", "Asking directions")
3. HTMX sends the request; chat interface loads without a full page reload
4. Backend retrieves the system\_prompt from the scenarios table and initializes a new record in **learning\_sessions**

### Alternative Flow:

- ❖ 1a. Free-tier user has reached the daily limit → UC12 triggers; upgrade modal displayed, scenario load blocked

### Postconditions:

- ❖ Row created in learning\_sessions
- ❖ AI persona for the selected scenario loaded into context

## 11.3. Set Difficulty Level (UC3)

Allows the Learner to configure their current CEFR proficiency level before starting a session. The selected level persisted to the user profile and used by the backend to calibrate the complexity of AI-generated prompts and responses throughout the conversation.

**Primary Actor:** Learner

**Secondary Actor:** None



## Project 9: AI Language Learning Companion (IT, Data Science)

### Preconditions:

- ❖ UC1 completed

### Main Flow:

1. Learner selects a CEFR level (A1–C2) from a dropdown
2. Selected level saved to `users.proficiency_level`
3. Backend uses this value when assembling LLM prompts to adjust vocabulary and sentence complexity

### Alternative Flow:

- ❖ None

### Postconditions:

- ❖ **proficiency\_level** persisted in the database
- ❖ Subsequent AI prompts calibrated to the chosen difficulty level

## 11.4. Record Voice Input (UC4)

Allows the Learner to capture a spoken response using the browser's MediaRecorder API. The recorded audio is sent to the Django backend for processing. This is the entry point of the real-time conversation pipeline and triggers the transcription, AI response and feedback chain.

**Primary Actor:** Learner

**Secondary Actor:** None

### Preconditions:

- ❖ UC1 completed
- ❖ UC2 scenario selected
- ❖ UC3 difficulty level set
- ❖ Microphone permission granted by the browser

### Main Flow:

1. Learner presses the "Record" button
2. Browser activates the MediaRecorder API; live waveform indicator is displayed
3. Learner speaks their turn in the target language
4. Learner presses "Stop"
5. Audio file is sent to the Django backend via HTMX hx-post
6. Django verifies file format and size; UC5 is triggered

### Alternative Flow:

- ❖ 2a. Microphone permission denied → error banner displayed; text-input fallback offered
- ❖ 5a. File too large or wrong format → backend rejects with a user-friendly error message

### Postconditions:

## Project 9: AI Language Learning Companion (IT, Data Science)

- ❖ Audio file received and verified by Django
- ❖ UC5 pipeline initiated

### 11.5. Transcribe Speech (STT)(UC5)

Handles the conversion of the Learner's recorded audio into text using the OpenAI Whisper API. The resulting transcript is the foundation for both the AI response generation (UC6) and the phonetic pronunciation assessment (UC8), which are triggered in parallel once transcription is complete.

**Primary Actor:** Learner

**Secondary Actor:** OpenAI Whisper (STT)

**Preconditions:**

- ❖ Valid audio file received from UC4

**Main Flow:**

1. Django preprocesses the audio via pydub/librosa and verifies format and size
2. Audio payload forwarded to the OpenAI Whisper API
3. Returned transcript validated for completeness
4. UC6 (AI response) and UC8 (phonetic assessment) are triggered in parallel

**Alternative Flow:**

- ❖ 2a. Whisper timeout → user-friendly error within 10 seconds; text-input fallback displayed

**Postconditions:**

- ❖ Transcript text available in memory
- ❖ **user\_transcript** field ready to be saved to **interaction\_logs**

### 11.6. Generate AI Response (UC6)

Generates a contextually relevant conversational reply using an LLM (Claude or GPT-4). The backend assembles a structured prompt combining the transcript, scenario context, difficulty level and conversation history. The LLM returns a reply alongside grammar corrections and a grammar score, triggering the feedback and audio playback chain.

**Primary Actor:** Learner

**Secondary Actor:** LLM (Claude / GPT-4)

**Preconditions:**

- ❖ UC5 transcript available
- ❖ Scenario context and conversation history loaded

**Main Flow:**

1. Django assembles a structured prompt: transcript + scenario context + difficulty level + conversation history

## Project 9: AI Language Learning Companion (IT, Data Science)

2. Prompt sent to the LLM API
3. LLM returns: (a) conversational reply, (b) grammar corrections, (c) grammar score 0–100
4. UC7 (grammar feedback), UC9 (TTS audio), and UC14 (memory update) are triggered

### Alternative Flow:

- ❖ 2a. LLM timeout → retry once; if still failing, user-friendly error displayed within 10 seconds

### Postconditions:

- ❖ AI reply text stored in memory
- ❖ Grammar score and corrections ready for UC7
- ❖ UC9 and UC14 proceed with the generated response

## 11.7. Provide Grammar Feedback (UC7)

Displays real-time grammar corrections and a 0–100 grammar score to the Learner after each spoken turn. Feedback is injected directly into the chat interface via HTMX without a page reload, with incorrect tokens highlighted using Bootstrap utility classes.

**Primary Actor:** Learner

**Secondary Actor:** None

### Preconditions:

- ❖ UC6 completed with grammar correction payload

### Main Flow:

1. HTMX injects the feedback panel below the user's chat bubble
2. Corrected phrases highlighted; incorrect tokens marked in a contrasting colour using Bootstrap utility classes
3. Grammar score (0–100) displayed to the Learner

### Alternative Flow:

- ❖ None

### Postconditions:

- ❖ Feedback visible to the Learner
- ❖ detailed\_feedback JSONB field saved to interaction\_logs

## 11.8. Phonetic Pronunciation Assessment (UC8)

Evaluates the accuracy of the Learner's pronunciation by running a phonetic similarity algorithm on the transcript against the target words. Results are displayed alongside the grammar feedback panel, and flagged words are stored for later review in the progress dashboard.

**Primary Actor:** Learner

**Secondary Actor:** None

## Project 9: AI Language Learning Companion (IT, Data Science)

### Preconditions:

- ❖ UC5 transcript available

### Main Flow:

1. Django runs the phonetic similarity algorithm on the transcript against the target words
2. Pronunciation score and flagged problem words computed
3. Results injected via HTMX alongside the grammar feedback panel

### Alternative Flow:

- ❖ None

### Postconditions:

- ❖ Pronunciation score saved in `detailed_feedback` JSONB
- ❖ Mispronounced words are available for the word list in the UC10 dashboard

## 11.9. Play AI Voice Response (UC9)

Converts the AI's text reply into natural-sounding speech using the ElevenLabs TTS API and delivers it to the Learner through the browser's audio player. The audio plays automatically alongside the text bubble, allowing the Learner to both hear and read the AI response simultaneously.

**Primary Actor:** Learner

**Secondary Actor:** ElevenLabs (TTS API)

### Preconditions:

- AI text reply from UC6 ready

### Main Flow:

1. Django sends the AI text response to the ElevenLabs API
2. Realistic audio file generated; file path saved to the database
3. HTMX injects the `<audio>` element into the conversation thread
4. Browser auto-plays the AI response; manual replay controls also rendered for accessibility

### Alternative Flow:

- ❖ 1a. ElevenLabs timeout → text bubble displayed without audio; user can request manual replay

### Postconditions:

- ❖ `ai_audio_url` saved in `interaction_logs`
- ❖ Learner hears and reads the AI response simultaneously

### 11.10. View Progress Dashboard (UC10)

Provides the Learner with a personalized overview of their learning history, including session records, average grammar scores over time and a list of consistently mispronounced or misused words. The dashboard is server-rendered by Django and uses indexed database queries to ensure fast load times even as data grows.

**Primary Actor:** Learner

**Secondary Actor:** Supabase / PostgreSQL

**Preconditions:**

- ❖ UC1 completed
- ❖ At least one complete session in the system

**Main Flow:**

1. Learner navigates to the dashboard
2. Django server-renders the page querying **learning\_sessions** and **interaction\_logs**
3. System displays: session history, average grammar score over time, and a word list of consistently mispronounced or misused terms
4. Queries use B-Tree indexes (**idx\_sessions\_user\_id**, **idx\_logs\_session\_id**) to guarantee fast retrieval

**Alternative Flow:**

- ❖ 2a. No prior sessions → welcome message inviting the Learner to start their first practice session

**Postconditions:**

- ❖ Learners have full visibility of their progress over time
- ❖ Data isolated per user via Supabase RLS policies

### 11.11. Manage Subscription (UC11)

Allow the Learner to view their current subscription tier (Free or Premium). The active plan determines the daily usage limits enforced by UC12. Payment processing is out of scope for this phase; this use case covers plan visibility only.

**Primary Actor:** Learner

**Secondary Actor:** None

**Preconditions:**

- ❖ UC1 completed

**Main Flow:**

1. Learner navigates to the subscription page
2. System displays the current plan (Free / Premium)



## Project 9: AI Language Learning Companion (IT, Data Science)

3. Learners can review the differences between plans
4. **users.subscription\_plan** reflects the active tier

### Alternative Flow:

- ❖ None

### Postconditions:

- ❖ Subscription tier visible to the Learner
- ❖ Backend enforces usage limits via UC12

## 11.12. Enforce Usage Limits (UC12)

A system-level use case that intercepts UC2 and UC4 to verify whether the Learner is within their daily usage quota. Free-tier users are limited to five conversations per day. This use case extends UC2 and UC4; it is not initiated directly by the Learner.

**Primary Actor:** System

**Secondary Actor:** None

### Preconditions:

- ❖ User subscription tier known (**users.subscription\_plan**)

### Main Flow:

1. Before loading a scenario (UC2) or starting a recording (UC4), the backend checks the daily conversation count
2. Free Plan: limit of 5 conversations per day.
3. If the limit has not been reached, the flow continues normally
4. If the limit has been reached, an upgrade modal is displayed and the action is blocked

### Alternative Flow:

- ❖ 3a. Premium user → no restriction, flow continues uninterrupted

### Postconditions:

- ❖ Free-tier users cannot exceed the daily limit
- ❖ Premium users proceed without restriction

## 11.13. Save Session Data (UC13)

An automatic system process that persists all data generated during a conversation turn to the `interaction_logs` table in PostgreSQL. This includes audio URLs, transcripts, AI responses, grammar scores, pronunciation scores and detailed JSONB feedback. It is triggered internally after each turn and requires no direct action from the Learner.

**Primary Actor:** System (*automatic process — no direct Learner initiation*)

**Secondary Actor:** Supabase / PostgreSQL

## Project 9: AI Language Learning Companion (IT, Data Science)

### Preconditions:

- ❖ Interaction turns completed

### Main Flow:

1. Django automatically saves to `interaction_logs`: audio URL, transcript, AI response text, AI audio URL, grammar score, pronunciation score, and detailed JSONB feedback
2. Foreign key links the record to the corresponding **learning\_session**
3. RLS policy ensures only the owning user can read or write the record

### Alternative Flow:

- ❖ 1a. Supabase connection failure → data held in memory; retry with backoff until saved

### Postconditions:

- ❖ All interaction data correctly persisted
- ❖ Data available for UC10 dashboard queries
- ❖ B-Tree indexes guarantee fast retrieval

## 11.14. Maintain Conversation Memory (UC14)

An automatic system process that retrieves the ordered history of prior conversation turns at the start of each new turn and injects it into the LLM prompt. This ensures the AI maintains contextual awareness across the entire session, producing coherent and continuous dialogue rather than isolated responses.

**Primary Actor:** System (*automatic process — no direct Learner initiation*)

**Secondary Actor:** Supabase / PostgreSQL

### Preconditions:

- ❖ Active **learning\_session**
- ❖ At least one prior record in **interaction\_logs** for the session

### Main Flow:

1. At the start of each new turn, Django automatically queries **interaction\_logs** for the current session, indexed by **session\_id**
2. Previous turns assembled into an ordered conversation history
3. History injected into the LLM prompt in UC6 to maintain context across turns

### Alternative Flow:

- ❖ None

### Postconditions:

- ❖ AI responds coherently to earlier exchanges within the session
- ❖ Conversation feels continuous and natural to the Learner

### 11.15. Diagram 1 - Authentication & Configuration

This diagram covers all use cases that must be completed before a conversation session can begin, including user authentication, scenario selection, difficulty configuration and subscription management. UC12 operates as a background system process that extends UC2 and UC4 to enforce daily usage limits for Free-tier users. Since UC4 is defined in Diagram 2, a cross-diagram note is included to maintain the accuracy of the extend relationship. **Figure 2**

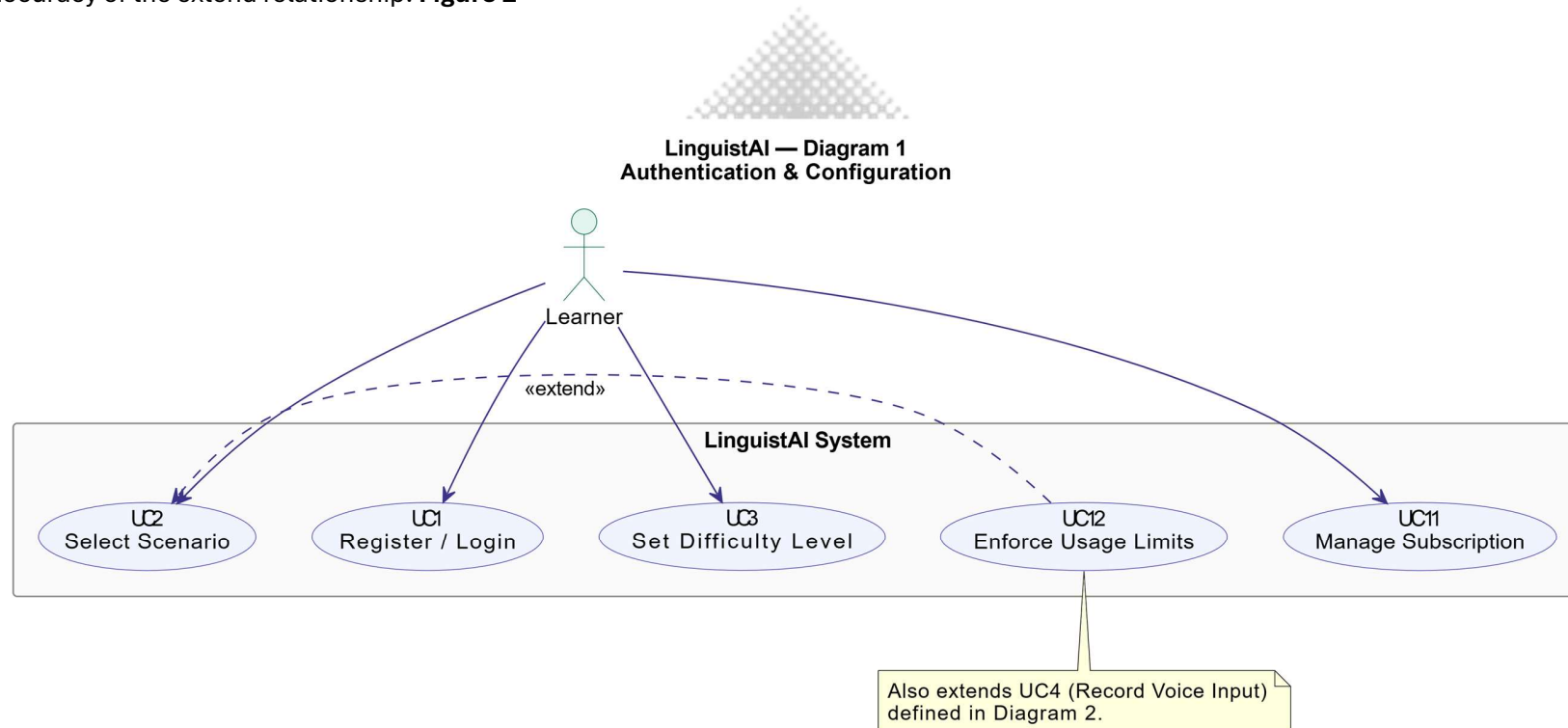


Figure 2 - Authentication & Configuration Diagram

### 11.16. Diagram 2 - Conversation Core

This diagram captures the complete real-time conversation pipeline, from voice recording to AI-generated feedback and audio playback. The flow is structured as a chain of include relationships starting from UC4, triggering transcription, AI response generation, grammar feedback, phonetic assessment, voice playback and conversation memory in sequence. Three secondary actors (OpenAI Whisper, the LLM and ElevenLabs) each handle a distinct stage of the pipeline. **Figure 3**

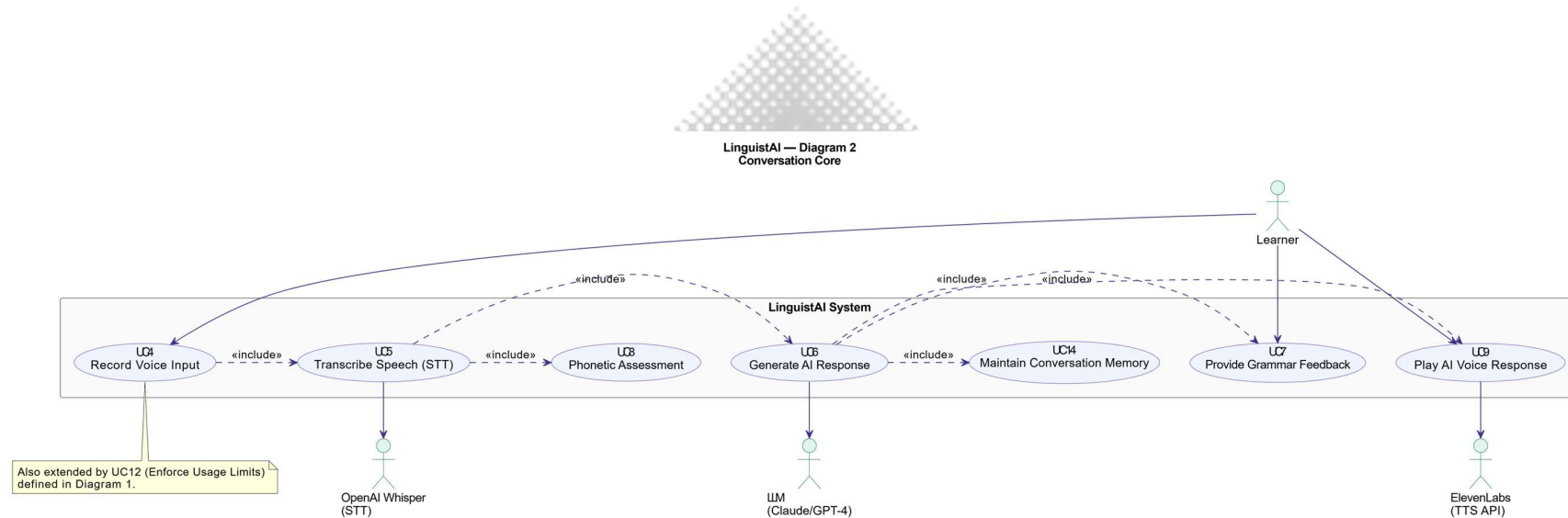


Figure 3 - Conversation Core Diagram

### 11.17. Diagram 3 - Persistence & Progress

This diagram covers the data persistence and progress tracking layer of the system. UC13 and UC14 are automatic background processes that save interaction data and maintain conversation memory after each turn, requiring no direct Learner action. UC10 is the only Learner-facing use case, providing access to session history, grammar scores and a personalized list of mispronounced words. **Figure 4**

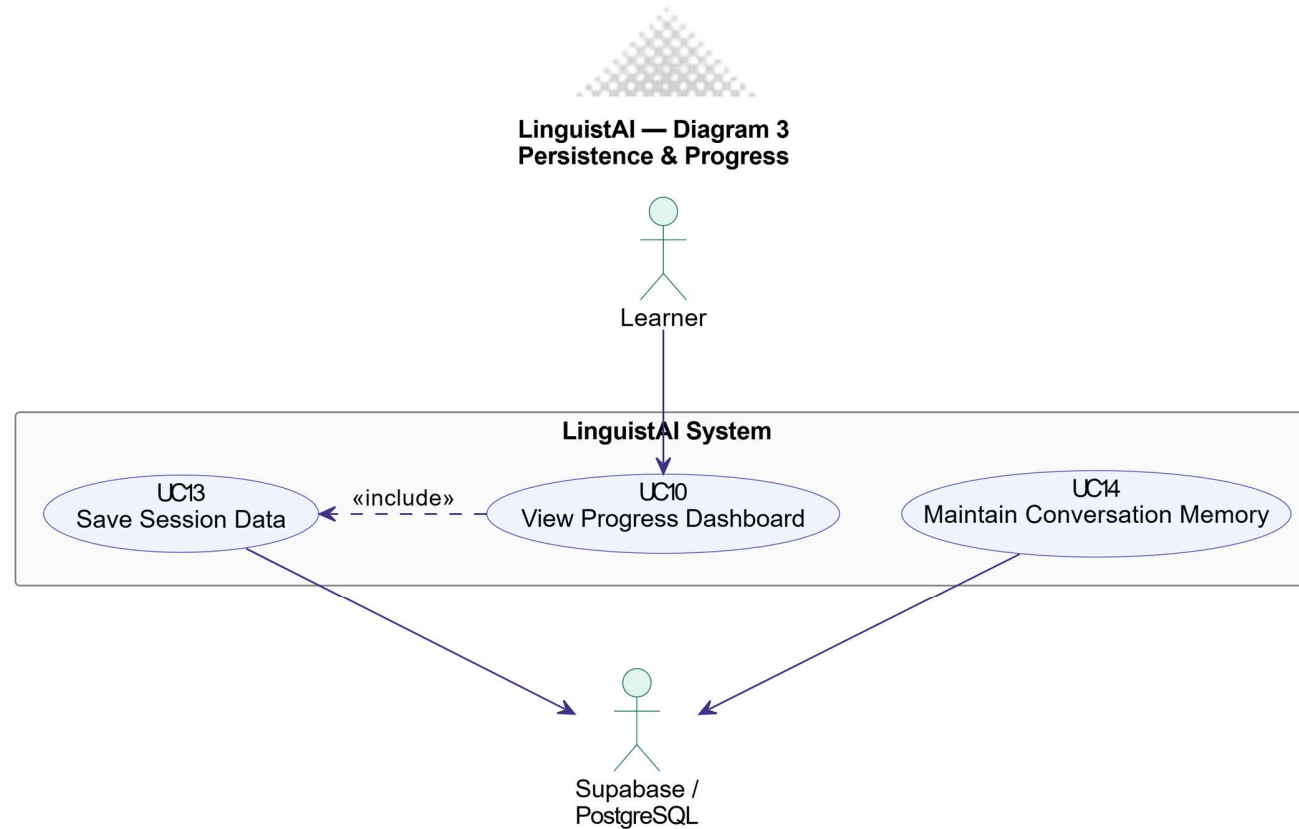


Figure 4 - Persistence & Progress Diagram

## 12. Sequence Diagram - Written by Camilo & Quoc

### 12.1. Sequence Diagram 1 - Authentication & Configuration

This diagram illustrates the message flow for the Authentication and Configuration layer, covering login, scenario selection, difficulty configuration, subscription retrieval and usage limit enforcement. All interactions are synchronous request-response cycles between the Browser, Django, and PostgreSQL, with no external APIs involved. **Figure 5**

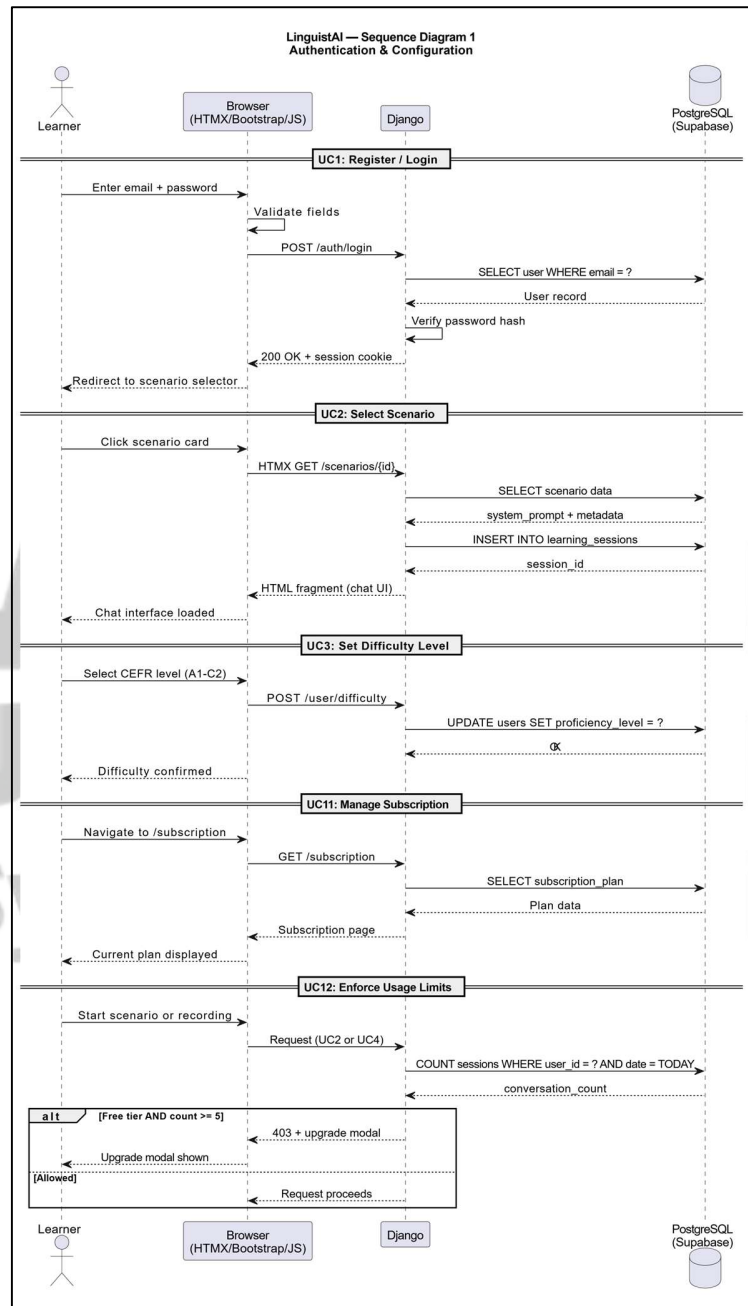


Figure 5 - Authentication & Configuration Diagram



## 12.2. Sequence Diagram 2 - Conversation Core

This diagram traces the full message flow of a single conversation turn across six participants (Browser, Django, Whisper, the LLM, ElevenLabs and PostgreSQL). It shows the sequential dependency between use cases throughout the pipeline, including conversation history retrieval before each LLM call. Alternative flows are included for each external API to demonstrate graceful degradation in the event of a timeout or failure. **Figure 6**

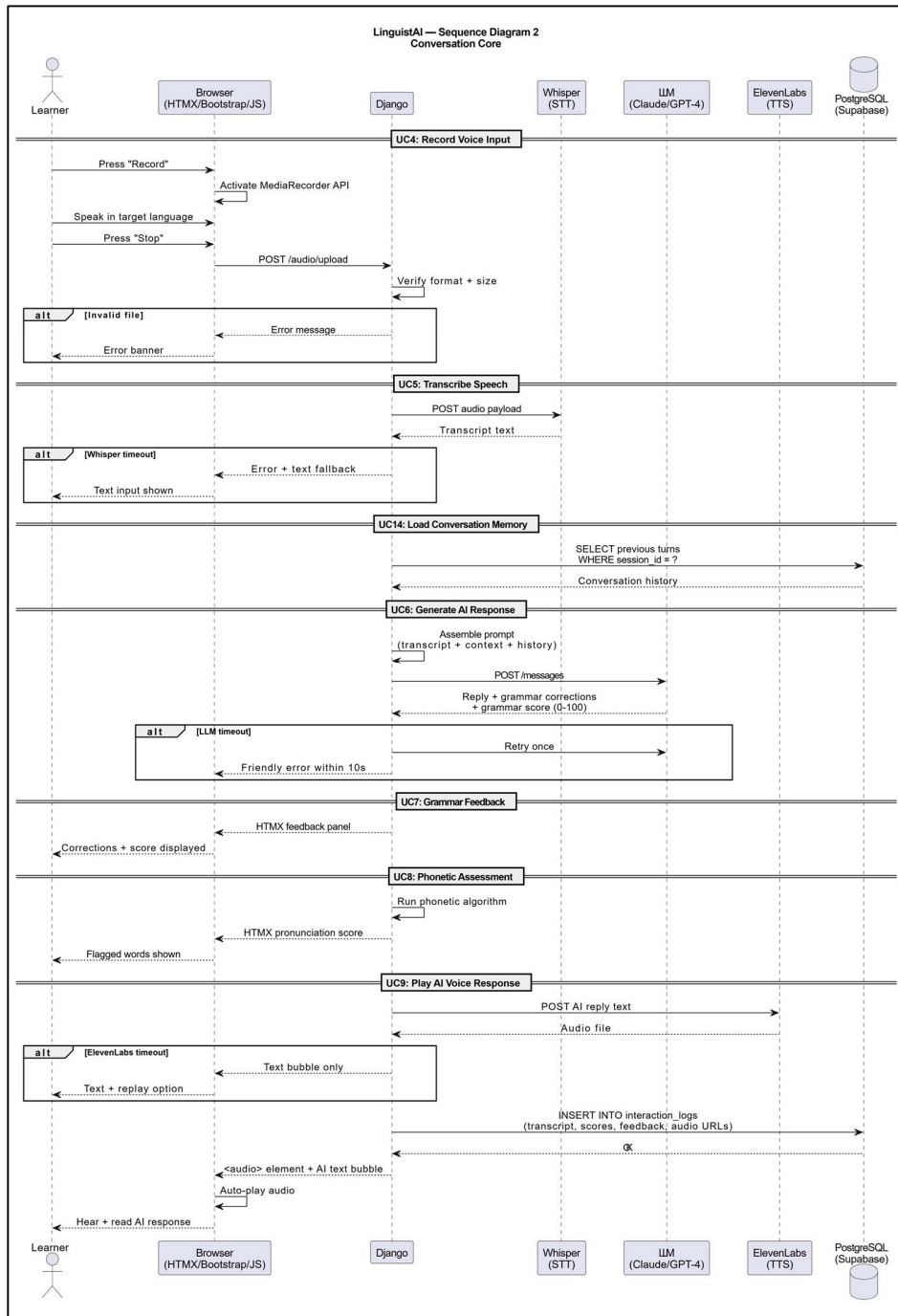


Figure 6 - Conversation Core

### 12.3. Sequence Diagram 3 - Persistence & Progress

This diagram covers the data persistence, and progress retrieval flows that run alongside the conversation pipeline. UC13 and UC14 are presented as automatic system processes, showing how interaction data is saved and how conversation history is assembled before each new turn. The UC10 sequence demonstrates the two-stage database query and server-side computation used to build the progress dashboard. **Figure 7**

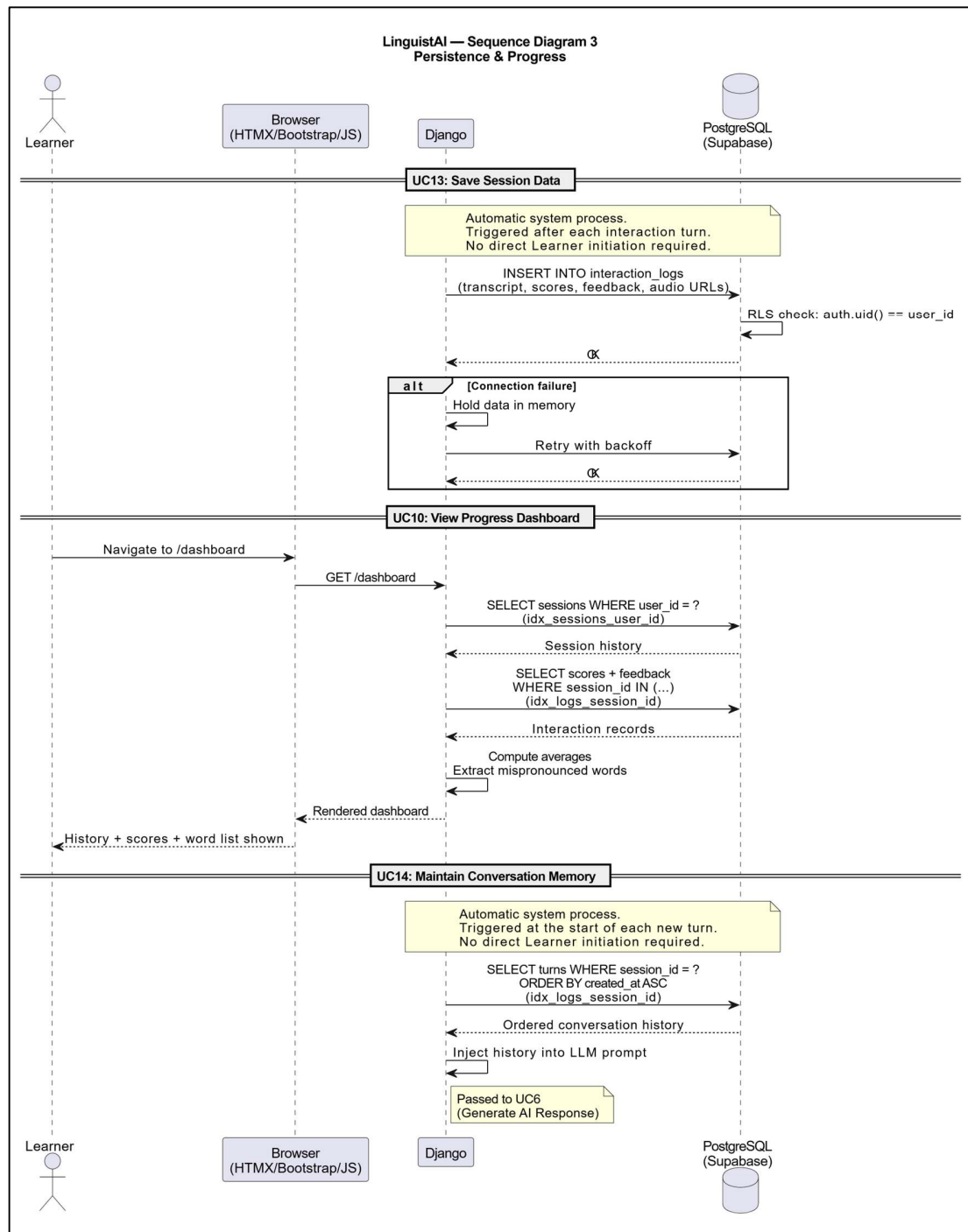


Figure 7 - Persistence & Progress Diagram

## 13. Pseudocode - Written by Quoc

### Purpose

This pseudocode provides a high-level, language-agnostic representation of the core backend algorithm and system workflows for the LinguistAI platform. It demonstrates the logical flow, data handling, and seamless integration between the Django server-end, PostgreSQL database, HTMX front-end and external AI APIs (OpenAI Whisper, GPT-4, ElevenLabs).

### Conventions

- ❖ Keywords: Standard control flow and structural keywords are written in UPPERCASE (e.g., FUNCTION, IF, THEN, RETURN).
- ❖ Function Names: Written in PascalCase (e.g., ProcessAudioInput, AnalyzeAndRespond).
- ❖ ID fields: Suffixes are consistent across the database parameters (e.g., user\_id, scenario\_id, session\_id, log\_id).
- ❖ Variables: Written in snake\_case to clearly differentiate local variables from global functions and keywords (e.g., hashed\_password, user\_transcript, ai\_responsejson).
- ❖ Database Operations: Abstracted using DB prefixes to clearly represent standard PostgreSQL CRUD operations without getting bogged down in SQL syntax (e.g., DB\_SELECT, DB\_INSERT, DB\_UPDATE).
- ❖ API Responses: Standard HTTP status codes are utilized to represent the successful or failed outcome of a server-side block (e.g., HTTP\_200\_OK, HTTP\_401\_UNAUTHORIZED, HTTP\_503\_SERVICE\_UNAVAILABLE).

## 13.1. Authentication and Session Management

### 13.1.1. Sign Up

```
FUNCTION SignUp(email, username, password):
  IF UserExists(email) OR UserExists(username) THEN
    RETURN HTTP_400_BAD_REQUEST("User already exists")
  END IF

  hashed_password = HashPassword(password)
  default_proficiency = "Beginner"

  new_user_id = DB_INSERT("Users", {email, username, hashed_password, default_proficiency})
  RETURN HTTP_201_CREATED("Account successfully created", new_user_id)
END FUNCTION
```

**Note:** Handles secure user registration by checking for duplicates, encrypting the password to meet security standards, setting a default learning level, and saving the record.

### 13.1.2. Sign In

```
FUNCTION SignIn(email, password):
  user_record = DB_SELECT("Users").WHERE("email" == email)

  IF NOT user_record OR NOT VerifyPassword(password, user_record.hashed_password) THEN
    RETURN HTTP_401_UNAUTHORIZED("Invalid credentials")
  END IF

  session_token = GenerateSecureSessionToken(user_record.id)
```

## Project 9: AI Language Learning Companion (IT, Data Science)

```
RETURN HTTP_200_OK("Login successful", session_token)
END FUNCTION
```

**Note:** Verifies user credentials and generates a secure session token. This token acts as a digital pass to maintain the authenticated state across the application.

### 13.1.3. Require Auth (API Helper)

```
FUNCTION RequireAuth(request):
    token = ExtractTokenFromHeader(request)

    IF NOT IsValidToken(token) THEN
        REDIRECT_TO("/login")
    END IF

    RETURN GetUserIdFromToken(token)
END FUNCTION
```

**Note:** Acts as security middleware. It intercepts incoming API requests to validate the session token, ensuring only logged-in learners can access their voice data and histories.

## 13.2. Learning Context & Profile Management

### 13.2.1. Update Proficiency Profile

```
FUNCTION UpdateProficiencyProfile(user_id, new_level):
    valid_levels = ["Beginner", "Intermediate", "Advanced"]

    IF new_level NOT IN valid_levels THEN
        RETURN HTTP_400_BAD_REQUEST("Invalid proficiency level")
    END IF

    DB UPDATE("Users").SET("proficiency_level" = new_level).WHERE("id" == user_id)
    RETURN HTTP_200_OK("Proficiency profile updated")
END FUNCTION
```

**Note:** Allows the system to dynamically adjust the user's language level. This variable is later passed to the AI to customize the difficulty of the conversation.

### 13.2.2. Initialize Learning Scenario

```
FUNCTION InitializeSession(user_id, scenario_id):
    scenario = DB_SELECT("Scenarios").WHERE("id" == scenario_id)

    IF NOT scenario THEN
        RETURN HTTP_404_NOT_FOUND("Scenario not found")
    END IF

    session_id = DB_INSERT("Learning_Sessions", {user_id, scenario_id, started_at: NOW()})
    RETURN HTTP_200_OK("Session started", session_id, scenario.video_url)
END FUNCTION
```

**Note:** Starts a new practice run by fetching the selected situational context (e.g., "Ordering food") and generating a unique **session\_id** to track the upcoming interactions.

## 13.3. Core Voice Processing & AI Generation

### 13.3.1. Process Audio Input (STT)

```
FUNCTION ProcessAudioInput(audio_blob):
    IF NOT ValidateFileFormat(audio_blob, ["wav", "webm"]) THEN
        RETURN HTTP_415_UNSUPPORTED_MEDIA("Invalid audio format")
    END IF
```

## Project 9: AI Language Learning Companion (IT, Data Science)

```
secure_audio_url = UploadToCloudStorage(audio_blob, "/temp_user_audio/")
user_transcript = CallOpenAIWhisperAPI(secure_audio_url)

RETURN user_transcript, secure_audio_url
END FUNCTION
```

**Note:** Performs strict data verification on the incoming audio file, uploads it to cloud storage, and uses the OpenAI Whisper API to accurately transcribe the spoken words into text.

### 13.3.2. Generate Conversation & Feedback Analysis

```
FUNCTION AnalyzeAndRespond(user_transcript, scenario_prompt, user_proficiency):
    llm_compiled_prompt = BuildPrompt(scenario_prompt, user_proficiency, user_transcript)

    // Request structured JSON from GPT-4
    ai_response_json = CallGPT4API(llm_compiled_prompt)

    reply_text = ai_response_json.get("reply_text")
    grammar_fixes = ai_response_json.get("grammar_corrections")
    pronunciation_score = ai_response_json.get("pronunciation_score")

    RETURN reply_text, grammar_fixes, pronunciation_score
END FUNCTION
```

**Note:** The core logic engine. It combines the transcript, scenario context, and user level into one prompt, requesting both a conversational reply and real-time educational feedback from GPT-4.

### 13.3.3. Voice Generation (TTS)

```
FUNCTION GenerateAIVoice(ai_reply_text):
    ai_audio_blob = CallElevenLabsTTSAPI(ai_reply_text)
    secure_ai_audio_url = UploadToCloudStorage(ai_audio_blob, "/ai_responses/")

    RETURN secure_ai_audio_url
END FUNCTION
```

**Note:** Sends the AI's generated text response to the ElevenLabs API to produce a natural, human-like voice audio file, completing the two-way spoken conversation loop.

## 13.4. Storage Policy and History

### 13.4.1. Interaction Logging

```
FUNCTION LogInteraction(session_id, transcript, user_audio_url, reply_text, ai_audio_url,
metrics_json):
    DB_INSERT("Interaction_Logs", {
        session_id: session_id,
        user_transcript: transcript,
        user_audio_url: user_audio_url,
        ai_response_text: reply_text,
        ai_audio_url: ai_audio_url,
        detailed_feedback: metrics_json,
        created_at: NOW()
    })
    RETURN True
END FUNCTION
```

**Note:** Fulfills database auditing requirements by permanently saving the full footprint of a single conversational turn into the PostgreSQL database using a flexible JSON structure.

### 13.4.2. Fetch Session History

```
FUNCTION FetchSessionHistory(session_id, user_id):  
    // RLS (Row Level Security) validation  
    session_owner = DB_SELECT("Learning_Sessions").WHERE("id" == session_id).get("user_id")  
    IF session_owner != user_id THEN  
        RETURN HTTP_403_FORBIDDEN("Access denied")  
    END IF  
  
    logs = DB_SELECT("Interaction_Logs").WHERE("session_id" == session_id).ORDER_BY("created_at", ASC)  
    RETURN HTTP_200_OK(logs)  
END FUNCTION
```

**Note:** Retrieves past chat logs to populate the UI. It includes Row Level Security (RLS) validation to guarantee users can absolutely only view records they own.

## 13.5. Export & Utilities

### 13.5.1. Export Learning Report

```
FUNCTION ExportSessionReport(session_id):  
    logs = DB_SELECT("Interaction_Logs").WHERE("session_id" == session_id)  
  
    formatted_pdf = ConvertLogsToPDFFormat(logs)  
    download_link = GenerateTemporaryDownloadLink(formatted_pdf)  
  
    RETURN HTTP_200_OK(download_link)  
END FUNCTION
```

**Note:** A utility function that compiles a user's session history and AI grammar feedback into a downloadable PDF format for offline study and review.

### 13.5.2. Generate HTMX UI Fragment

```
FUNCTION RenderChatBubbleFragment(ai_audio_url, grammar_fixes, pronunciation_score):  
    html_fragment = LoadTemplate("partials/chat_bubble.html")  
  
    html_fragment.Inject(ai_audio_url)  
    html_fragment.Inject(grammar_fixes)  
    html_fragment.Inject(pronunciation_score)  
  
    RETURN html_fragment  
END FUNCTION
```

**Note:** Demonstrates seamless integration. It generates a lightweight HTML fragment populated with the AI's response data, which HTMX injects directly into the chat UI without refreshing the page.

## 13.6. API Error Handling & Rate Limiting

### 13.6.1. Enforce Rate Limits

```
FUNCTION CheckRateLimit(user_id):  
    interactions_last_hour = DB_COUNT("Interaction_Logs").WHERE("user_id" == user_id AND "created_at" > NOW() - 1 HOUR)  
  
    IF interactions_last_hour >= 50 THEN  
        RETURN HTTP_429_TOO_MANY_REQUESTS("Rate limit exceeded. Please try again later.")  
    END IF  
  
    RETURN True  
END FUNCTION
```



**Note:** A critical non-functional security measure. It prevents API abuse and controls infrastructure costs by restricting users to a maximum of 50 voice interactions per hour.

### 13.6.2. Safe External API Calling with Fallback

```
FUNCTION SafeAPICall(api_function, payload, max_retries=3):
    attempt = 0
    WHILE attempt < max_retries DO
        TRY
            response = api_function(payload)
            IF response.status == 200 THEN
                RETURN response.data
            END IF
        CATCH TimeoutError OR APIError AS e
            attempt = attempt + 1
            Wait(ExponentialBackoff(attempt))
        END TRY
    END WHILE

    RETURN HTTP_503_SERVICE_UNAVAILABLE("AI processing is temporarily down.")
END FUNCTION
```

**Note:** Implements an exponential backoff retry mechanism. This ensures the platform handles temporary external API outages (OpenAI/ElevenLabs) gracefully without crashing the user's session.

## 13.7. Storage Management & Garbage Collection

### 13.7.1. Scheduled Audio Cleanup (CRON Job)

```
FUNCTION CleanupOldAudioFiles():
    retention_threshold = NOW() - 30 DAYS
    expired_logs = DB_SELECT("Interaction_Logs").WHERE("created_at" < retention_threshold AND
"audio_deleted" == False)

    FOR EACH log IN expired_logs DO
        TRY
            DeleteFromCloudStorage(log.user_audio_url)
            DeleteFromCloudStorage(log.ai_audio_url)

            DB_UPDATE("Interaction_Logs")
            .SET("user_audio_url" = NULL, "ai_audio_url" = NULL, "audio_deleted" = True)
            .WHERE("id" == log.id)

        CATCH Error e
            LogSystemError("Failed to delete audio for log: " + log.id)
        END TRY
    END FOR

    RETURN "Cleanup complete"
END FUNCTION
```

**Note:** A maintenance script designed to optimize system resources. It automatically deletes heavy audio files older than 30 days from cloud storage while preserving the lightweight text data for the user's history.

## 14. UI Design - Written by Win & Quoc

### 14.1. Home Page

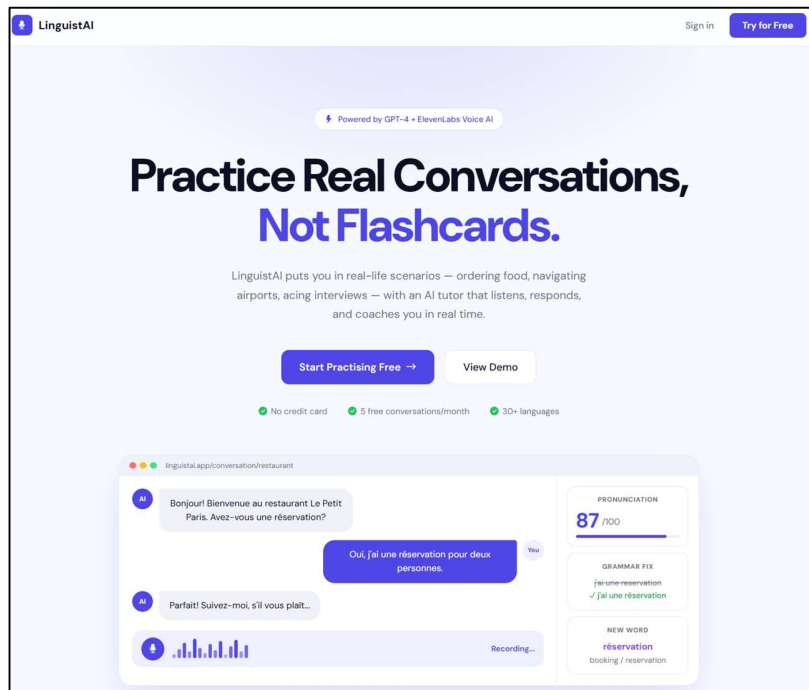


Figure 8 - Home Page

### 14.2. Sign in Page

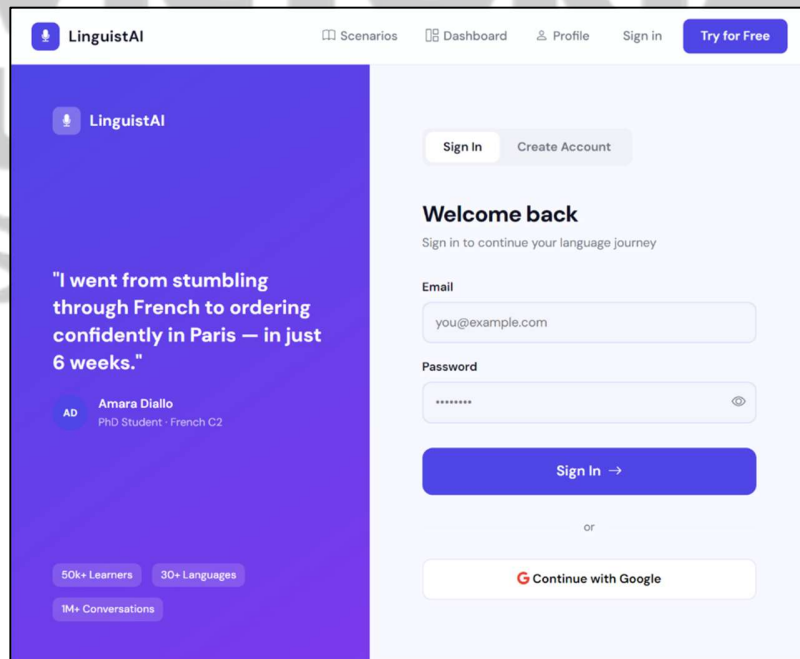


Figure 9 - Sign in Page

## 14.3. Dashboard Page

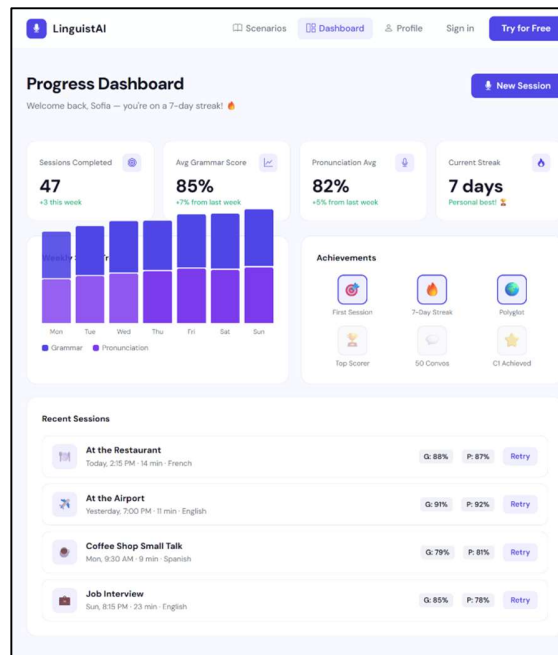


Figure 10 - Dashboard Page

## 14.4. Scenarios Library Page

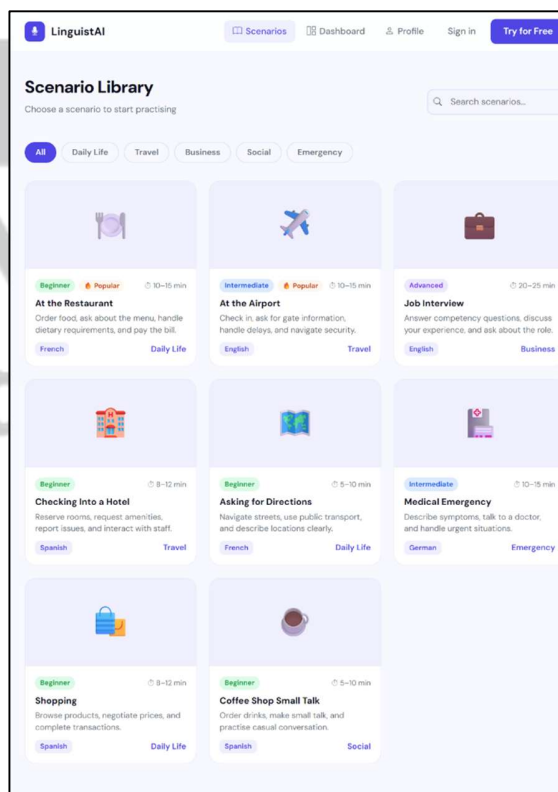


Figure 11 - Scenarios Library Page

14.5. Individual Scenario Section Page

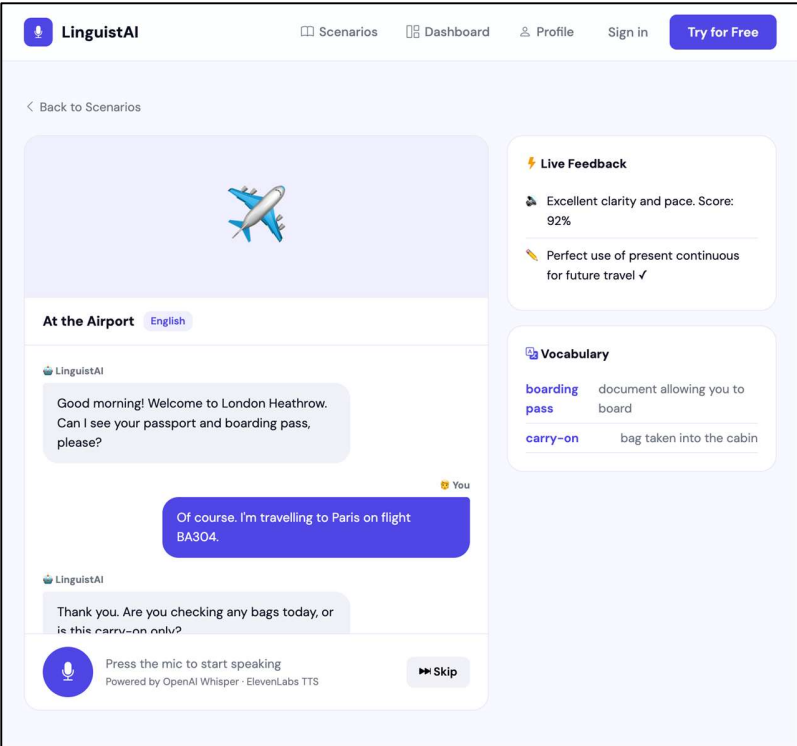


Figure 12 - Individual Scenario Section Page

14.6. Profile Page

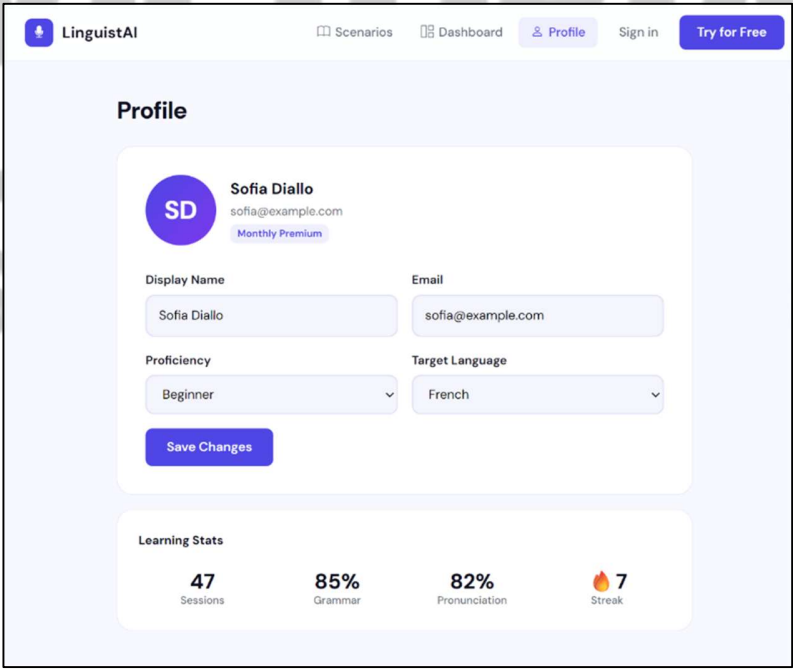


Figure 13 - Profile Page

## 15. Risk & Resource Management - Written by Jeison

### 15.1. Resource Allocation Framework

Development of lifecycle management for **LinguistAI** will focus on the planning of human resources and technical system tools. Members with relevant skills will be grouped in technical domains so that their work can be done within the strict university block parameters. Also, cloud resources will be monitored so that costs can be managed in terms of Australian Dollars (AUD). **Table 14**

| Resource Category  | Assigned Resource Item                   | Primary Operational Function               | Weekly Allocation  | Planned Contribution / Project Scope                                                                                                                                                                                                                                                                                                                                                 |
|--------------------|------------------------------------------|--------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Human Capital      | s8200193 - Win, Htay Hlaing              | Frontend & UI Developer                    | 12 hours / week    | <ul style="list-style-type: none"> <li>* Base Interfaces: Build responsive Bootstrap 5 templates for all platform pages.</li> <li>* Microphone Logic: Implement the JS MediaRecorder API to capture user voice input.</li> <li>* HTMX Integration: Replace page reloads with HTMX fragment swaps for dynamic UI updates.</li> </ul>                                                  |
| Human Capital      | s8131699 - Quoc Anh, Nguyen              | Backend & Data Integration Lead            | 12 hours / week    | <ul style="list-style-type: none"> <li>* Database Setup: Architect Supabase PostgreSQL tables in 3NF with UUIDs and Row Level Security (RLS).</li> <li>* Audio Pipeline: Develop server-side logic using pydub/librosa to process audio files.</li> <li>* AI Integrations: Securely script API calls to OpenAI Whisper, Claude/GPT-4, and ElevenLabs.</li> </ul>                     |
| Human Capital      | s8159573 – Salazar Diaz, Camilo          | Systems Flow & QA Lead                     | 12 hours / week    | <ul style="list-style-type: none"> <li>* Conversation Memory: Build the logic to query and append past interaction_logs to the LLM prompt.</li> <li>* Error Handling: Implement fallback messages and exponential backoff retries for external APIs.</li> <li>* NFR Testing: Conduct latency tests and Locust load testing for concurrent users.</li> </ul>                          |
| Human Capital      | s8187769 - Aguirre Angulo, Jeison Andres | Business Logic & Operations                | 12 hours / week    | <ul style="list-style-type: none"> <li>* Usage Limits: Write interceptor logic to restrict Free Tier users to 5 daily conversational turns.</li> <li>* Dashboard Analytics: Develop server-side calculations for average scores and mispronounced word tracking.</li> <li>* Resource Management: Deploy CRON jobs for garbage collection to minimize cloud storage costs.</li> </ul> |
| Academic Oversight | Dr Sardar Farhad                         | Project Supervisor & Convenor              | Academic Audits    | Pedagogical validation, rubric evaluation checkpoints, milestone assessment sign-off.                                                                                                                                                                                                                                                                                                |
| Cloud Compute      | Supabase Database Layer                  | PostgreSQL transactional session tracking  | Free Tier Package  | Permanent cloud backend server data hosting layer.                                                                                                                                                                                                                                                                                                                                   |
| AI Processing      | OpenAI Whisper API Keys                  | Processing and transcribing user speech    | \$0.0085 AUD/min   | Speech-to-Text audio parsing microservice.                                                                                                                                                                                                                                                                                                                                           |
| AI Evaluation      | Anthropic Claude 3.5 Sonnet              | Conversational evaluation & feedback       | \$4.25 AUD/M input | Contextual reasoning and grammar feedback engine.                                                                                                                                                                                                                                                                                                                                    |
| AI Voice Engine    | ElevenLabs TTS API Keys                  | Synthesizing dynamic character audio loops | \$0.21 AUD/K chars | Speech generation voice model output layer.                                                                                                                                                                                                                                                                                                                                          |

Table 14 - Project Resource Allocation Matrix

## 15.2. Risk Analysis and Mitigation Matrix

Operating an interactive web application dependent on external multi-API data streams introduces technical, operational, and financial volatility. Project threats are assessed using a standard likelihood and impact framework (scored 1 to 5), yielding a calculated Risk Rating (Likelihood X Impact) and a clear assignment of responsibility. **Table 15**

| Risk ID | Risk Category | Threat Description                                                                                                                            | Likelihood (1-5) | Impact (1-5) | Risk Rating (1-25) | Core Technical & Team Mitigation Strategy                                                                                                     | Risk Owner |
|---------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------|--------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------|
| RSK-01  | Technical     | <b>API Gateway Latency:</b> Multi-service sync lags across Whisper, Claude, or ElevenLabs break real-time conversation dynamics.              | 4                | 4            | <b>16 (High)</b>   | Deploy asynchronous server workers; utilize front-end HTMX fragment swaps to stream chat text before full TTS audio payload transfers finish. | Camilo     |
| RSK-02  | Financial     | <b>AI Cost Escalation:</b> High volume testing loops trigger sudden API consumption fee spikes in Australian Dollars (AUD).                   | 3                | 4            | <b>12 (Medium)</b> | Build hard transactional rate limits and automated context-token filters directly into user profile validation query structures.              | Win        |
| RSK-03  | Security      | <b>Prompt Injection Attacks:</b> Malicious client inputs overwrite predefined linguistic scenario parameters or leak internal system prompts. | 3                | 5            | <b>15 (High)</b>   | Enforce rigid system role constraints, sanitize text buffers, and run validation checks before forwarding strings to external API endpoints.  | Win        |
| RSK-04  | Operational   | <b>Schedule Slippage / Delay:</b> Complex pronunciation modules cause delays, threatening university submission boundaries.                   | 2                | 5            | <b>10 (Medium)</b> | Maps all sprint goals to a Teamhood Gantt chart. Tri-weekly Discord reviews ensure trailing items are identified and reassigned.              | Jeison     |
| RSK-05  | Security      | <b>Data Pipe Leakage:</b> Unsecured data routes compromise historical user chat logs or personal authorization profiles.                      | 2                | 5            | <b>10 (Medium)</b> | Establish strict Row Level Security (RLS) policies on Supabase; secure cloud connection profiles with JSON Web Token (JWT) filters.           | Quoc Anh   |
| RSK-06  | Operational   | <b>Academic Misalignment:</b> Implemented features accidentally drift from supervisor expectations or explicit rubric boundaries.             | 2                | 4            | <b>8 (Low)</b>     | Uses Teamhood Gantt tracking to align milestones directly with Criteria guidelines prior to bi-weekly audits.                                 | Jeison     |

Table 15 - Risk Analysis and Mitigation Matrix



## 16. Timeline & Milestones - Written by Jeison

## 16.1. NIT3003 IT Capstone Project 1

**Figure 14** presents the NIT3003 IT Capstone Project 1 Gantt chart, capturing the complete project roadmap from the proposal opening stage to the final oral presentation. Each scheduled task is defined by clear operational boundaries, assigned individual accountability, and synchronized milestone evaluation points. This sequential framework ensures steady project momentum and the structured delivery of all major components.

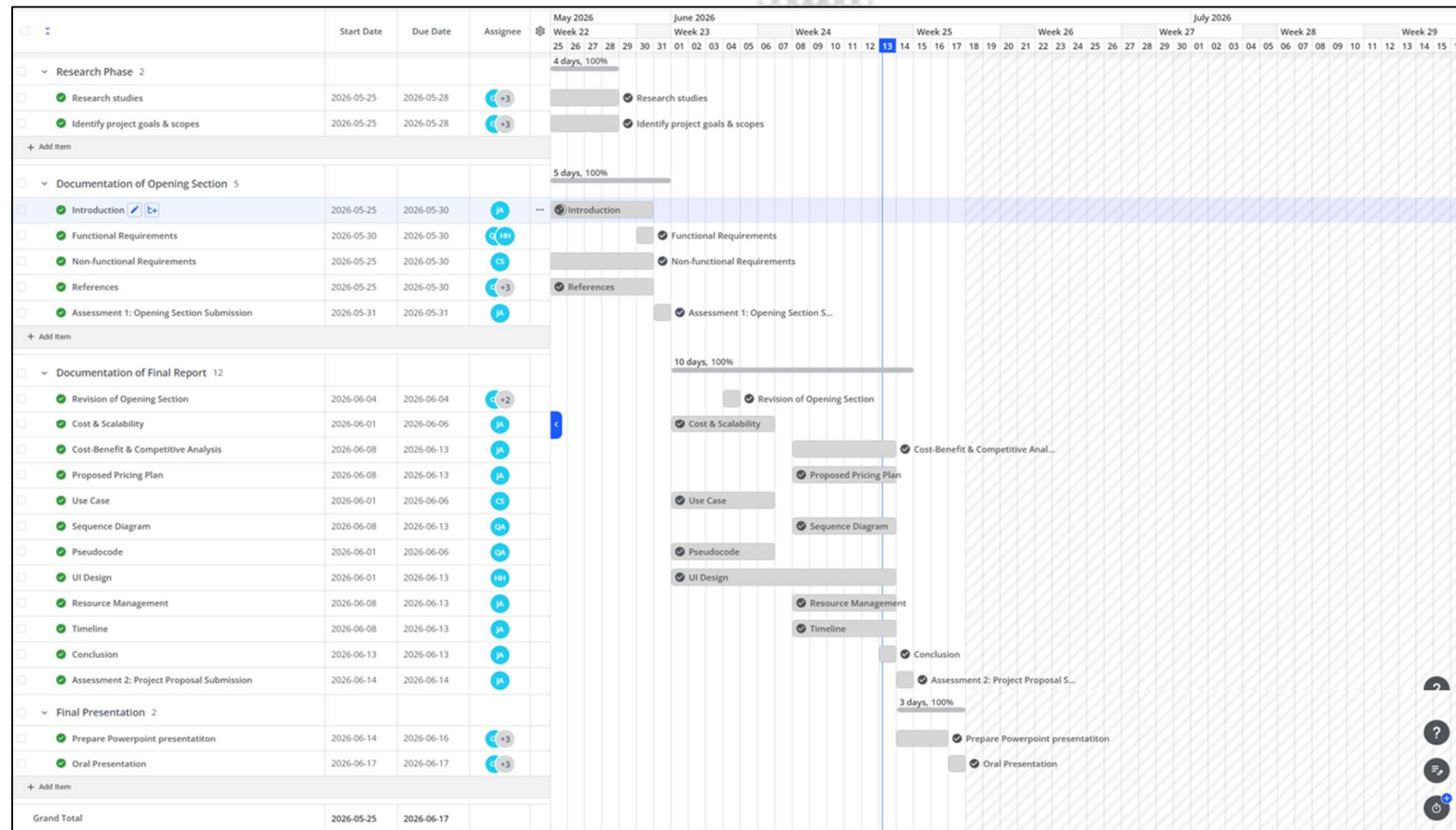


Figure 14 - NIT3003  
IT Capstone Project  
Gantt Chart

16.2. Project Implementation

Figure 15 & 16 presents the project implementation Gantt chart, tracking the software development phases from initial design to release readiness. The workflow defines clear milestones for integrating backend logic with the cloud database layer, deploying user authentication, establishing core AI features, and running system audit protocols. This methodical progression ensures all essential parameters are fully tested and functional before the team transitions into IT Capstone Project 2.

16.2.1. Part A - Project Implementation Gantt Chart

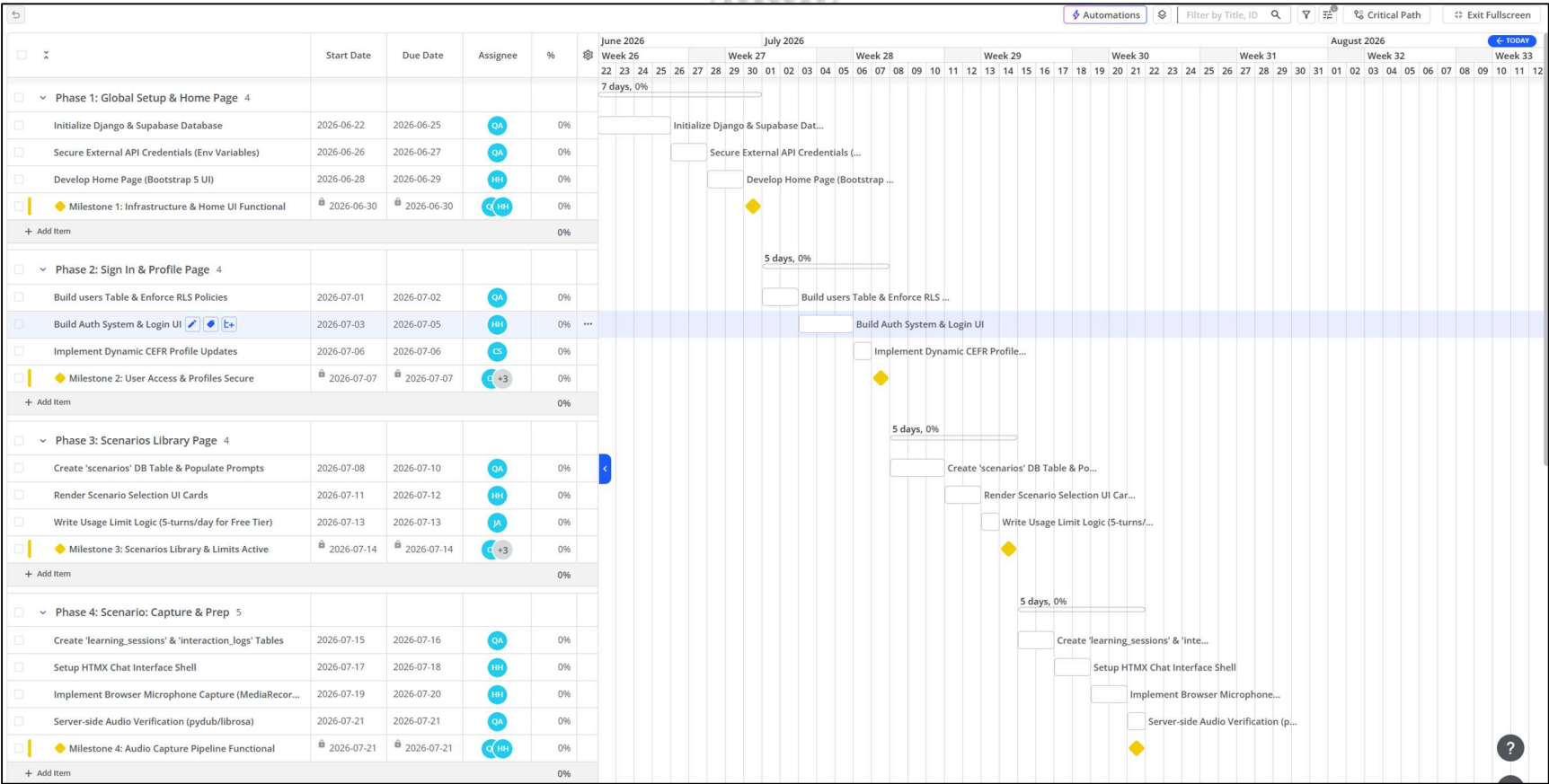


Figure 15 - Part A - Project Implementation Gantt Chart

## 16.2.2. Part B - Project Implementation Gantt Chart

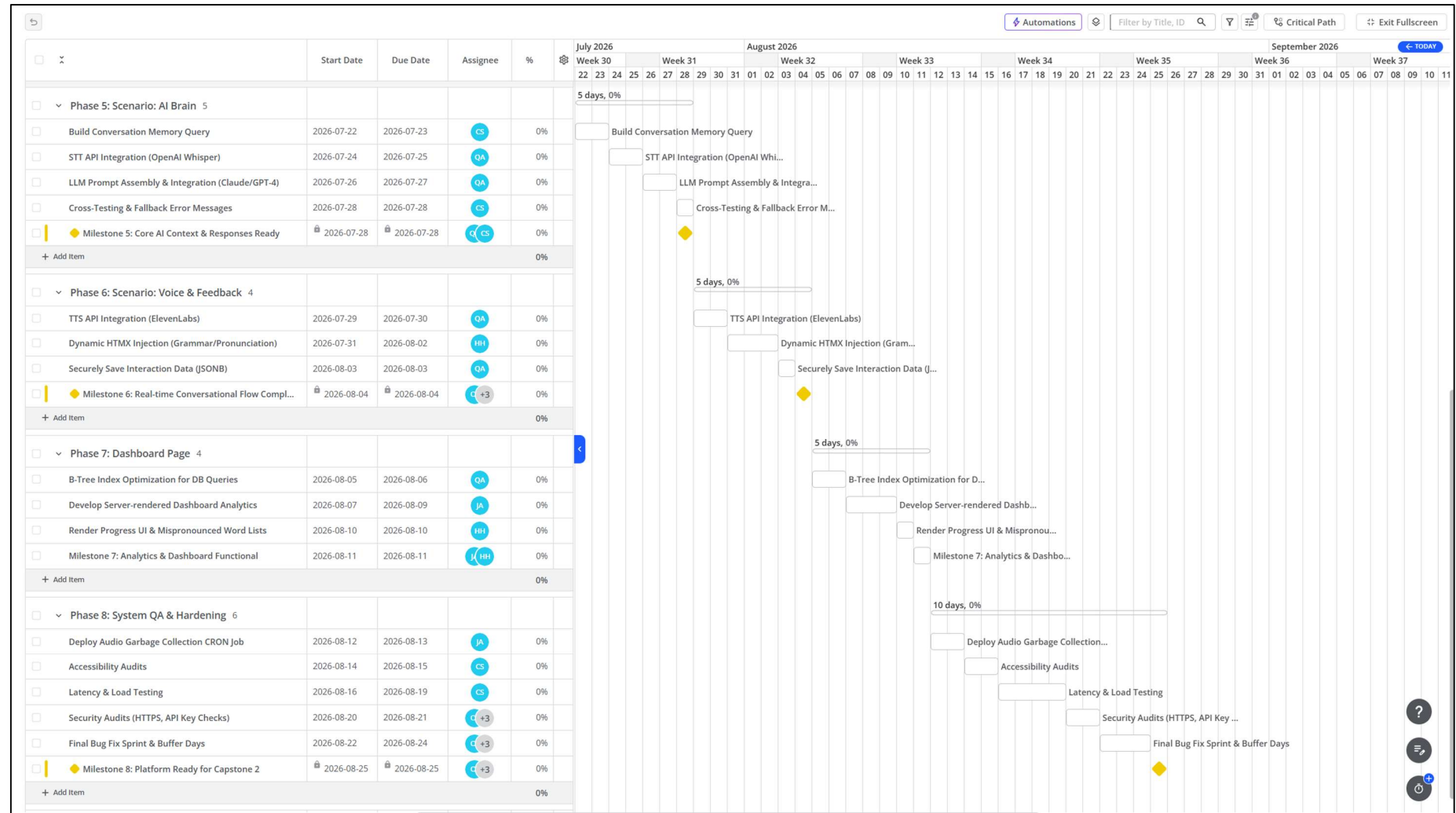


Figure 16 - Part B - Project Implementation Gantt Chart

16.3. NIT3004 IT Capstone Project 2

**Figure 17** highlights the sequential workflow for Capstone Project 2, prioritising milestone evaluation, documentation, and handover processes. The project timeline establishes overlapping tasks for engineering execution, continuous diary tracking, and rigorous final report preparation. This methodical structure guides the project to a close, leading directly to the final oral presentation stage.

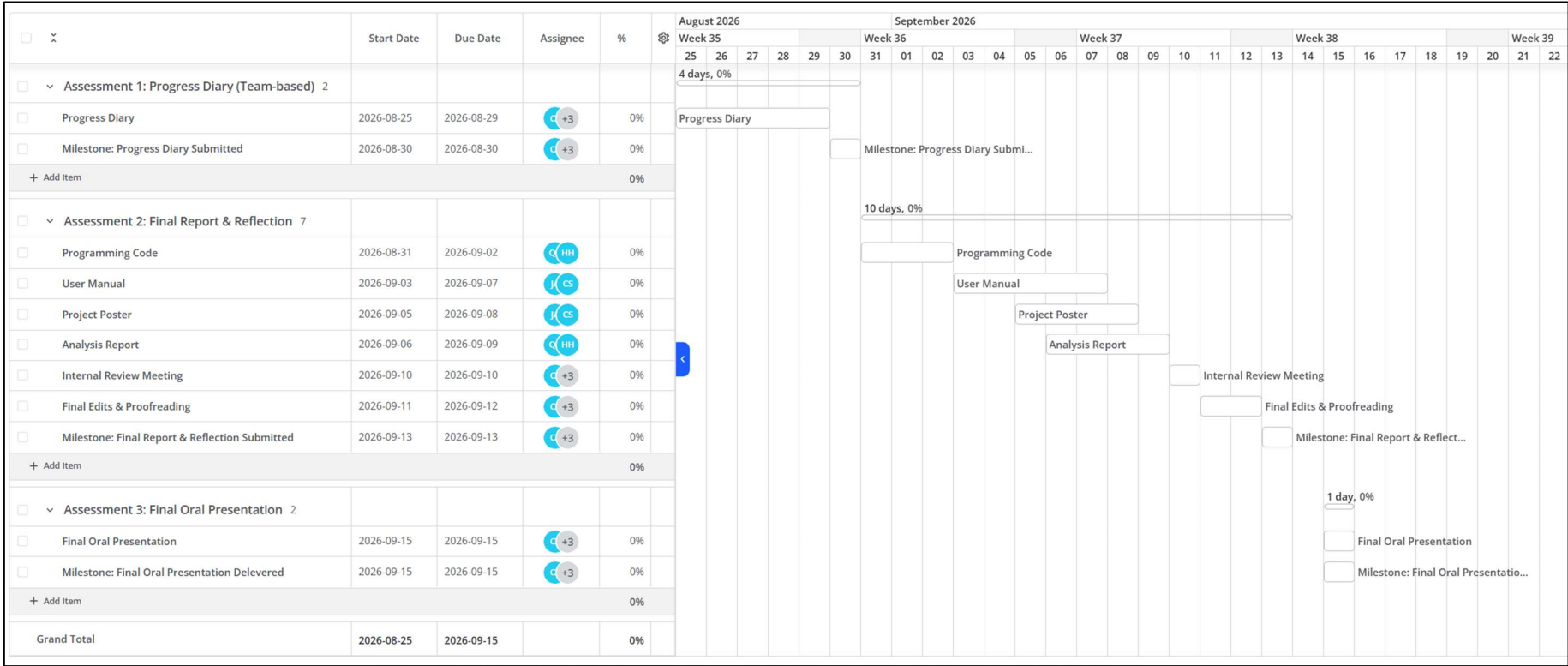


Figure 17 - NIT3004 IT Capstone Project 2 Gantt Chart

## 17. Conclusion - Written by Jeison

The introduction of LinguistAI is a significant response to a major challenge that the global language learning community has long faced: exhausting students' passive-level knowledge without enabling them to speak spontaneously or to express their thoughts actively. Indeed, traditional, heavily gamified platforms often focus on text repetition, memorization metrics, and keeping engagement streaks alive, but these methods do not mimic the dynamic and unpredictable nature of real-world oral communication. On the other hand, live tutoring marketplaces are excellent for practical conversational training, but they represent not only financial barriers to learners but also the possibility of causing performance anxiety.

### Technical Architecture & Integration

LinguistAI removes such hurdles by offering a scalable and purely web-based solution created with Django, HTMX, and a secure Supabase PostgreSQL database layer. The framework supports the seamless integration of generative technologies, leveraging:

- ❖ **OpenAI Whisper** for speech-to-text transcription.
- ❖ **Anthropic Claude 3.5 Sonnet** and **GPT-4** for in-depth contextual reasoning.
- ❖ **ElevenLabs** for voice synthesis.

### Commercial Viability & Value Proposition

The platform's market deviance being completely free of the 30% digital storefront distribution commission—boasts excellent unit economics. It utilizes a cost-efficient method to scale users through strict usage limitations on the free acquisition plan. Yet, it remains financially sustainable and very competitively priced with the Monthly Premium Plan at **\$29.99 AUD**.

Additionally, it meets highly demanding functional and non-functional specifications. It goes beyond Row Level Security to deliver extremely fast response latency and low-pressure, context-specific roleplay scenarios, giving the learner an affordable, 24/7 immersion environment that they can truly rely on to build real, communicative competence.

## References:

- Anthropic (2024). Claude API documentation. <https://docs.anthropic.com>
- Anthropic. (2024). *Claude API documentation*. <https://docs.anthropic.com>
- Australian Government (1992). Disability Discrimination Act 1992. <https://www.legislation.gov.au/Details/C2022C00209>
- Australian Government — Attorney-General's Department (2023). Privacy Act 1988. Federal Register of Legislation. <https://www.legislation.gov.au/Details/C2023C00237>
- BigSky Software (2024). HTMX documentation. <https://htmx.org/docs/>
- Bootstrap Team (2024). Bootstrap 5 documentation. <https://getbootstrap.com/docs/5.3/>
- Carson, C. (2024). *htmx - high power tools for HTML*. HTMX Documentation. <https://htmx.org/docs/>
- Carson, C. (2024). *htmx - high power tools for HTML*. HTMX Documentation. <https://htmx.org/docs/>
- Cooljugator. (2025, March 28). Pimsleur review 2025: How effective is this audio-based language learning method? Cooljugator Blog. <https://cooljugator.com/blog/pimsleur-review/>
- Copycat Cafe. (2026, June 5). *Duolingo Max review 2026: Is it worth \$168/year?* <https://copycatcafe.com/blog/duolingo-max>
- Cristiano, M. (2025, February 21). Full Rosetta Stone review: Effective legacy program but lacks interactive features. FluentU Blog. <https://www.fluentu.com/blog/reviews/rosetta-stone/>
- Django Software Foundation (2024). Django documentation (5.0): Authentication and password management. <https://docs.djangoproject.com/en/5.0/topics/auth/passwords/>
- Django Software Foundation. (2021). Security in Django. Django Documentation. <https://docs.djangoproject.com/en/3.2/topics/security/>
- Django Software Foundation. (2025). Django documentation (v5.2). <https://docs.djangoproject.com/en/5.2/>
- EdTech Innovation Hub (2026). Why most language learning apps fail to deliver long-term fluency. <https://www.edtechinnovationhub.com/news/why-most-language-learning-apps-fail-to-deliver-long-term-fluency>
- EdTech Innovation Hub. (2026, April 28). *Why most language learning apps fail to deliver long-term fluency*. <https://www.edtechinnovationhub.com>



## Project 9: AI Language Learning Companion (IT, Data Science)

- ElevenLabs. (2024). API reference: Introduction. ElevenLabs Documentation. <https://elevenlabs.io/docs/api-reference/introduction>
- ElevenLabs. (2024). Text to Speech API Reference. ElevenLabs Developer Documentation. <https://elevenlabs.io/docs/api-reference/text-to-speech>
- ElevenLabs. (2024). Text to speech. ElevenLabs Documentation. <https://elevenlabs.io/docs/overview/capabilities/text-to-speech>
- ElevenLabs. (2024). API reference: Introduction. ElevenLabs Documentation. <https://elevenlabs.io/docs/api-reference/introduction>
- Elmasri, R., & Navathe, S. B. (2015). Fundamentals of database systems (7th ed.). Pearson.
- Godwin-Jones, R. (2021). Artificial intelligence and language learning: Bridging the gap. *Language Learning & Technology*, 25(3), 1–18. <https://doi.org/10.125/llt.25.3.2021>
- Godwin-Jones, R. (2021). Artificial intelligence and language learning: Bridging the gap. *Language Learning & Technology*, 25(3), 1–18. <https://doi.org/10.125/llt.25.3.2021>
- HelloTalk Blog (2026). Language Learning Apps Compared : Why 90% of Learners Still Can't Speak. <https://www.hellotalk.com/en/blog/language-learning-apps-compared-2026>
- HelloTalk. (2026). *Language learning apps compared: Why 90% of learners still can't speak*. HelloTalk Blog. <https://www.hellotalk.com>
- Languages Around the Globe (2026). 5 Major Disadvantages of Using Language Learning Apps. <https://latg.org/disadvantages-language-learning-apps/>
- Languages Around the Globe. (2026). *5 major disadvantages of using language learning apps*. <http://www.latg.org>
- Maggi, C. (2025). Why Language Apps Keep Failing You (And What Actually Works). Medium. Available at: <https://medium.com/@chiaralanguagefreak/why-language-apps-keep-failing-you-and-what-actually-works-6623dd7655e7>
- Maggi, C. (2025). *Why language apps keep failing you (and what actually works)*. Medium. <https://medium.com/@chiaralanguagefreak>
- MDN Web Docs. (2024). Django web application security. Mozilla Developer Network. [https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Extensions/Server-side/Django/web\\_application\\_security](https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Django/web_application_security)
- Mozilla Developer Network (MDN). (2024). *MediaRecorder API*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>

## Project 9: AI Language Learning Companion (IT, Data Science)

- Nielsen, J. (1993). Usability engineering. Morgan Kaufmann
- Norman, D. A. (2013). The design of everyday things (Revised ed.). Basic Books
- Office of the Australian Information Commissioner (2023). Australian Privacy Principles. OAIC. <https://www.oaic.gov.au/privacy/australian-privacy-principles>
- OpenAI. (2022). Introducing Whisper. <https://openai.com/index/whisper/>
- OpenAI. (2023). Introducing ChatGPT and Whisper APIs. <https://openai.com/index/introducing-chatgpt-and-whisper-apis/>
- OpenAI. (2024). Whisper model. OpenAI API Documentation. <https://developers.openai.com/api/docs/models/whisper-1>
- OpenAI. (2024). Speech to text - OpenAI API. OpenAI Platform Documentation. <https://platform.openai.com/docs/guides/speech-to-text>
- OpenAI. (2024). *Whisper model*. OpenAI API Documentation. <https://developers.openai.com/api/docs/models/whisper-1>
- OWASP Foundation (2021). OWASP Top Ten: The ten most critical web application security risks. OWASP. <https://owasp.org/www-project-top-ten/>
- Oxley, B. (2024). Language Learning AI Companion: Key Features and Technology Stack.
- PostgreSQL Global Development Group (2024). PostgreSQL 16 documentation. <https://www.postgresql.org/docs/16/>
- PostgreSQL Global Development Group. (2024a). 8.14. JSON Types. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/datatype-json.html>
- PostgreSQL Global Development Group. (2024a). 8.14. JSON Types. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/datatype-json.html>
- PostgreSQL Global Development Group. (2024b). 8.12. UUID Type. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/datatype-uuid.html>
- PostgreSQL Global Development Group. (2024c). 11.1. Introduction to Indexes. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/indexes-intro.html>
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*. <https://www.postgresql.org/docs/16/>
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. Proc. ICML 2023. <https://proceedings.mlr.press/v202/radford23a.html>
- Rosendo Ríos, V. (2026). Optimizing monetization strategies for generative AI firms: Implications for search engagement. Comillas Pontifical University Institutional Repository. [Analyzing freemium-to-paid conversion thresholds and AI tier monetization limits].

## Project 9: AI Language Learning Companion (IT, Data Science)

Rosendo Ríos, V. (2026). *Optimizing monetization strategies for generative AI firms: Implications for search engagement*. Comillas Pontifical University Institutional Repository.

Supabase. (2024). *Row Level Security*. Supabase Documentation. <https://supabase.com/docs/guides/auth/row-level-security>

Supabase. (2024). *Row Level Security*. Supabase Documentation. <https://supabase.com/docs/guides/auth/row-level-security>

W3C (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. <https://www.w3.org/TR/WCAG21/>

Zhang, C., Meng, Y., & Ma, X. (2024). Artificial intelligence in EFL speaking: Impact on enjoyment, anxiety, and willingness to communicate. *Journal of Educational Technology Systems*. Advance online publication. <https://doi.org/10.1177/00472395241238693>

Zhang, C., Meng, Y., & Ma, X. (2024). Artificial intelligence in EFL speaking: Impact on enjoyment, anxiety, and willingness to communicate. *Journal of Educational Technology Systems*, 53(2), 145-168. <https://doi.org/10.1177/00472395241238693>

**VICTORIA**  
**UNIVERSITY**  
Sydney Australia

## Appendix A – Tables

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Table 1 - Competitor Comparison. ....                                 | 6  |
| Table 2 - Frontend Functionality (UI) .....                           | 10 |
| Table 3 - Server-Side Logic Table .....                               | 11 |
| Table 4 - Users Table .....                                           | 12 |
| Table 5 – Scenarios Table.....                                        | 12 |
| Table 6 - Learning Sessions Table .....                               | 12 |
| Table 7 – Interaction Logs Table.....                                 | 13 |
| Table 8 - Non-Functional Requirements Table .....                     | 18 |
| Table 9 - Unit Cost Matrix for AI API Infrastructure (AUD) .....      | 19 |
| Table 10 - Comparative Analysis of Language Learning Frameworks. .... | 20 |
| Table 11 -12-Month Scaled Revenue and Cost Projections (AUD) .....    | 21 |
| Table 12 - Competitive Subscription and Feature Matrix (AUD) .....    | 23 |
| Table 13 - Use Case Diagram — LinguistAI .....                        | 24 |
| Table 14 - Project Resource Allocation Matrix.....                    | 47 |
| Table 15 - Risk Analysis and Mitigation Matrix.....                   | 48 |



## Appendix B – Figures

|                                                               |    |
|---------------------------------------------------------------|----|
| Figure 1 - Bypassing the Fluency Plateau .....                | 20 |
| Figure 2 - Authentication & Configuration Diagram .....       | 33 |
| Figure 3 - Conversation Core Diagram .....                    | 34 |
| Figure 4 - Persistence & Progress Diagram .....               | 35 |
| Figure 5 - Authentication & Configuration Diagram .....       | 36 |
| Figure 6 - Conversation Core.....                             | 37 |
| Figure 7 - Persistence & Progress Diagram .....               | 38 |
| Figure 8 - Home Page.....                                     | 44 |
| Figure 9 - Sign in Page .....                                 | 44 |
| Figure 10 - Dashboard Page .....                              | 45 |
| Figure 11 - Scenarios Library Page .....                      | 45 |
| Figure 12 - Individual Scenario Section Page .....            | 46 |
| Figure 13 - Profile Page .....                                | 46 |
| Figure 14 - NIT3003 IT Capstone Project Gantt Chart .....     | 49 |
| Figure 15 - Part A - Project Implementation Gantt Chart ..... | 50 |
| Figure 16 - Part B - Project Implementation Gantt Chart ..... | 51 |
| Figure 17 - NIT3004 IT Capstone Project 2 Gantt Chart.....    | 52 |

**VICTORIA**  
**UNIVERSITY**  
Sydney Australia